DAYANANDA SAGAR UNIVERSITY

Department of Computer Science & Engineering

AI-Driven Software Engineering & Project Management

ASSIGNMENT -2

Course Code: 24CS3502 · Unit 3 - Containerization with Docker

Docker Containers - Hands-on Assignment

Part A: Docker Engine in an Ubuntu VM (VMware) · Part B: Docker Desktop

Name	Tushar Jatti
USN	ENG24CS0265
Roll no,	61
Section	5D
Mode	Individual
Date of submission	31st October 2026 (31/10/26)

System Details

Item	Details
Host OS and version	Microsoft Windows, version 10.0.26300.9711 (as printed by the Command Prompt in SS B1.2); NVMe SSD
Host processor	13th Gen Intel(R) Core(TM) i7-13620H, 2.40 GHz base, 10 cores / 16 logical processors, virtualization enabled (Task Manager)
Host RAM	15.6 GB (Task Manager); Docker Desktop reports 7.4 GB available to containers
VMware version	VMware Workstation 17.5 (VT-x Enabled)
VM name and settings	eng24cs0265-vm · 4 GB RAM · 2 CPU cores · 25 GB disk · network NAT
Ubuntu version in the VM	Ubuntu 26.04.1 LTS (codename "resolute"), kernel Linux 7.0.0-38-generic, x86-64. Note: Ubuntu 26.04.1 LTS was used instead of 24.04 LTS.
VM user / hostname	tushar @ tushar-VMware-Virtual-Platform
Docker version in the VM	Docker Engine - Community 29.9.1 (API 1.56), containerd 2.4.2, runc 1.5.2, Docker Compose v5.6.0
Docker Desktop version	Docker Desktop 4.96.0 (Engine 29.9.1, WSL 2 backend, context desktop-linux)

The VM is named with my USN in lowercase (eng24cs0265-vm), and container, image and volume names carry the USN (for example eng24cs0265-web, eng24cs0265-flask-app:v1, eng24cs0265-data). Terminal screenshots also show the user name "tushar" in the prompt, and some show a "name / usn" comment line.

Background in brief

A container packages an application together with its code, runtime and libraries so it runs the same way on every machine. A virtual machine virtualizes whole hardware and runs a full guest OS, whereas containers share the host kernel and only isolate user space, so they are smaller (megabytes) and start in seconds. In Part A both are used together: VMware runs a full Ubuntu VM, and Docker runs lightweight containers inside it.

Part A - Docker inside an Ubuntu VM (VMware)

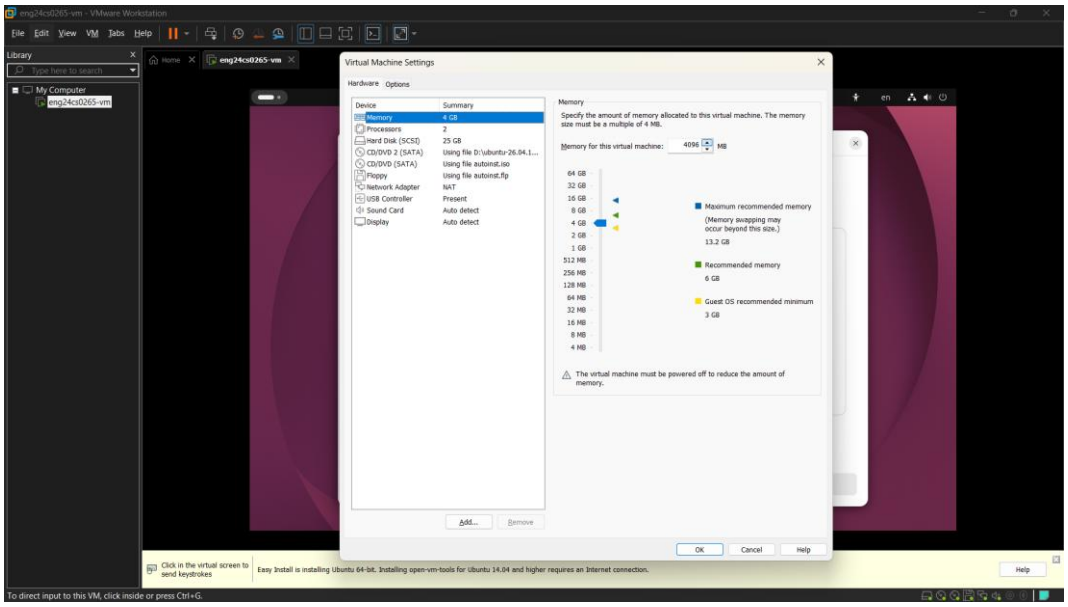
Experiment A1 - Install VMware and create an Ubuntu VM

Aim: Create a Linux virtual machine on the laptop.

```
hostnamectl
lsb_release -a

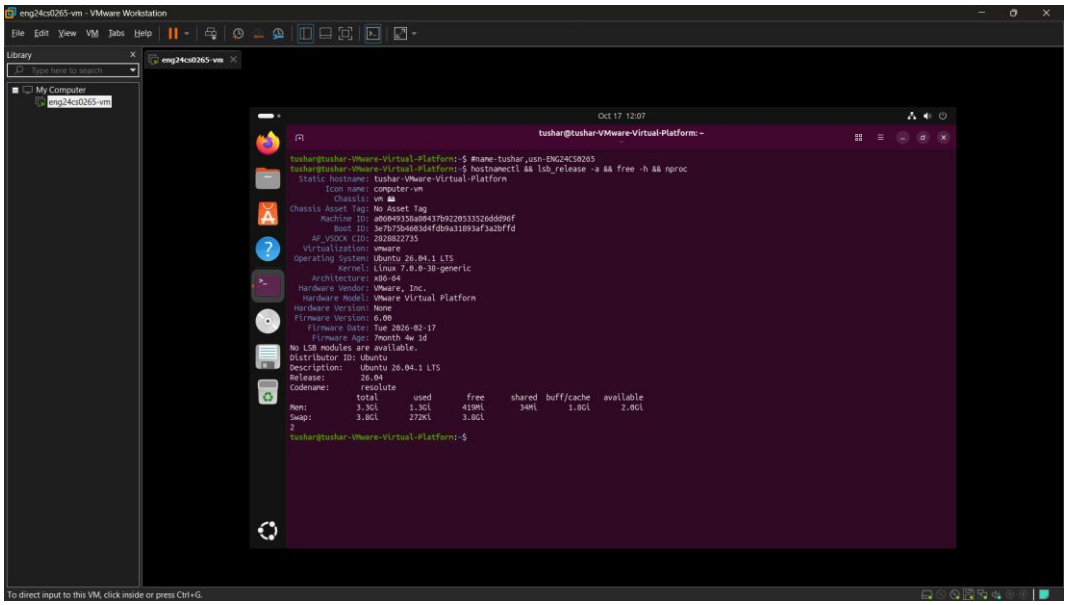
free -h
nproc
```

SS A1.1



VMware Workstation "Virtual Machine Settings" for the VM eng24cs0265-vm: 4 GB memory (4096 MB), 2 processors, 25 GB hard disk and a NAT network adapter.

SS A1.2



Ubuntu desktop running inside the VMware window (VM tab eng24cs0265-vm). The terminal shows my name/USN comment, hostnamectl, lsb_release -a (Ubuntu 26.04.1 LTS), free -h (3.3 GiB RAM) and nproc (2).

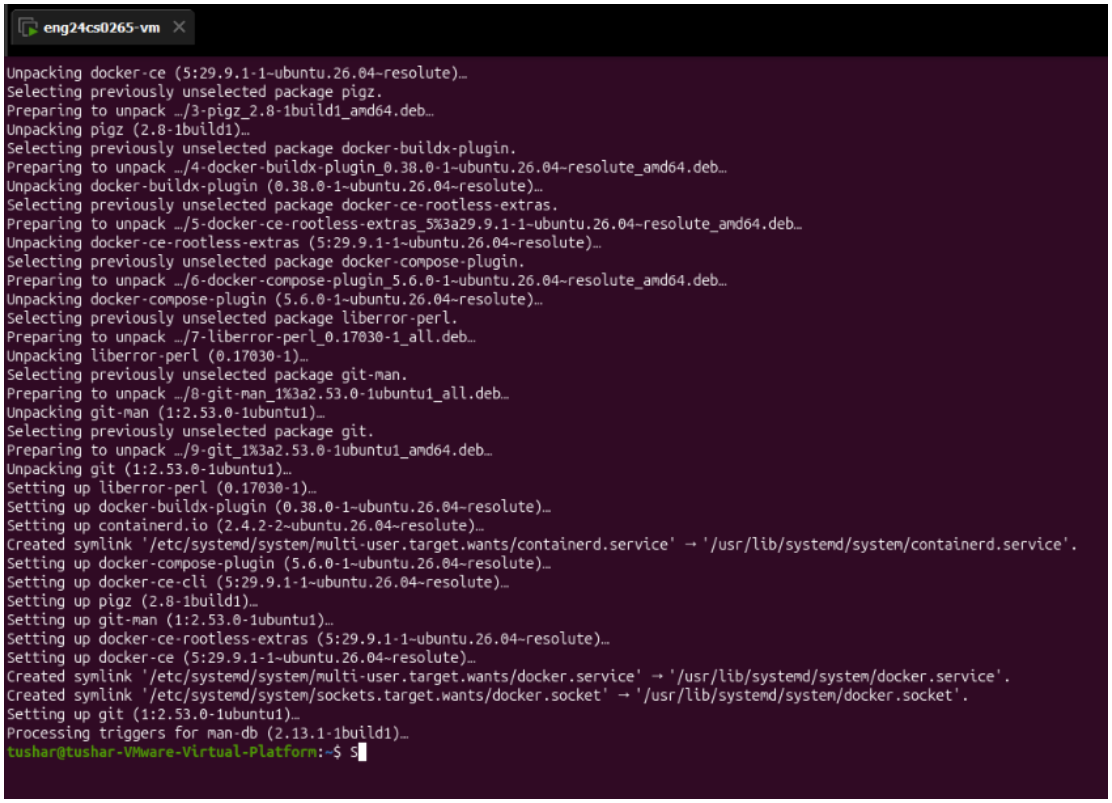
Experiment A2 - Install Docker Engine inside the VM

Aim: Install Docker from Docker's official repository and confirm it runs.

```
sudo apt update && sudo apt install -y ca-certificates curl   # repo set-up as in the sheet
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

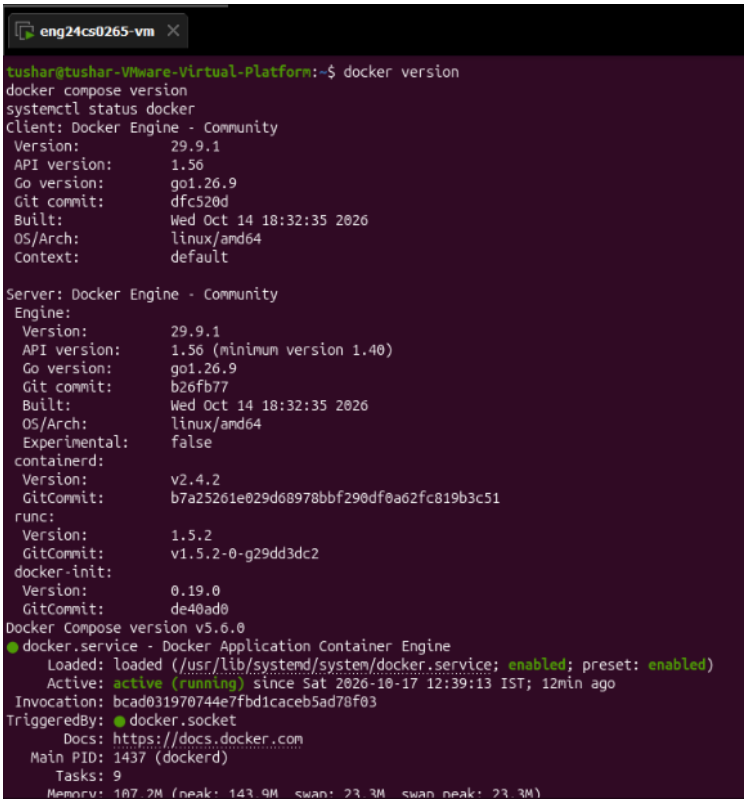
```
sudo usermod -aG docker $USER
docker --version; docker version; docker compose version; systemctl status docker
```

SS A2.1



Last lines of the apt install output: docker-ce, docker-ce-cli, containerd.io, docker-buildx-plugin and docker-compose-plugin are unpacked and set up, and the docker service and socket units are created.

SS A2.2



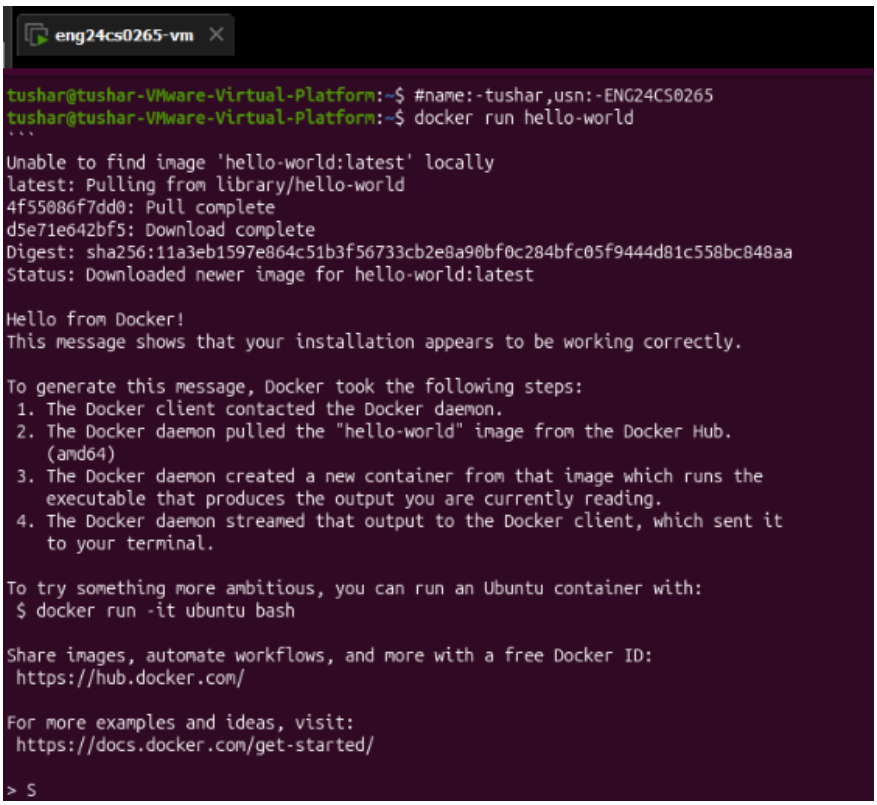
docker version showing both the Client and Server (Docker Engine 29.9.1), docker compose version v5.6.0, and systemctl status docker reporting "active (running)".

Experiment A3 - Run your first container

Aim: Understand what happens when you docker run an image you do not have.

```
docker run hello-world
```

SS A3.1



Full output of docker run hello-world: the image was not found locally, so it was pulled from Docker Hub, and then the container ran and printed "Hello from Docker!".

Write: List the 4 steps Docker says it performed?

Answer: The four steps Docker reports are: (1) the Docker client (the docker command I typed) contacted the Docker daemon, the background service; (2) the daemon pulled the hello-world image from Docker Hub because it was not on my machine; (3) the daemon created a new container from that image, which runs a small executable that produces the message; (4) the daemon streamed that output back to the client, which printed it in my terminal.

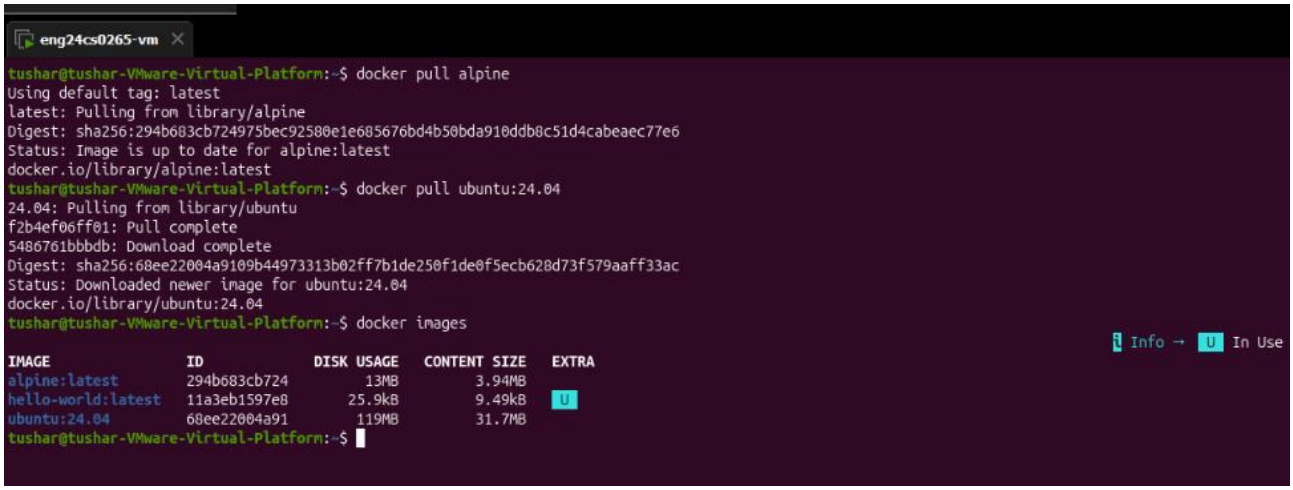
Experiment A4 - Working with images

Aim: Pull, list, inspect and compare images.

```
docker pull alpine
docker pull ubuntu:24.04

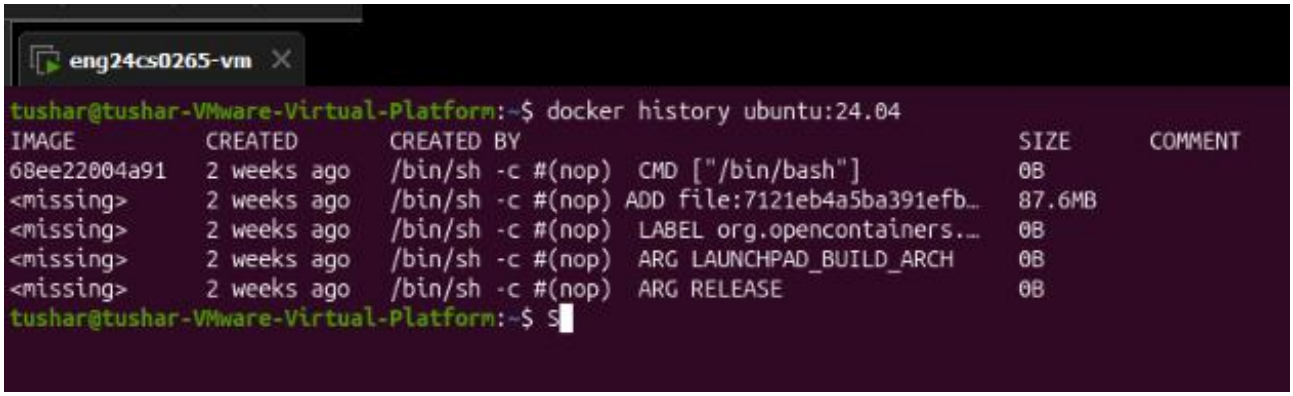
docker images
docker history ubuntu:24.04
```

SS A4.1



docker pull for alpine and ubuntu:24.04 followed by docker images: alpine 13 MB, hello-world 25.9 kB and ubuntu:24.04 119 MB of disk usage.

SS A4.2



docker history ubuntu:24.04 lists the image layers; almost all of the size is one 87.6 MB layer (the root filesystem), the others are 0 B metadata layers such as CMD, LABEL and ARG.

Write: Why is alpine so much smaller than ubuntu ? Compare the size of the ubuntu:24.04 image with the 25 GB disk you gave the VM.

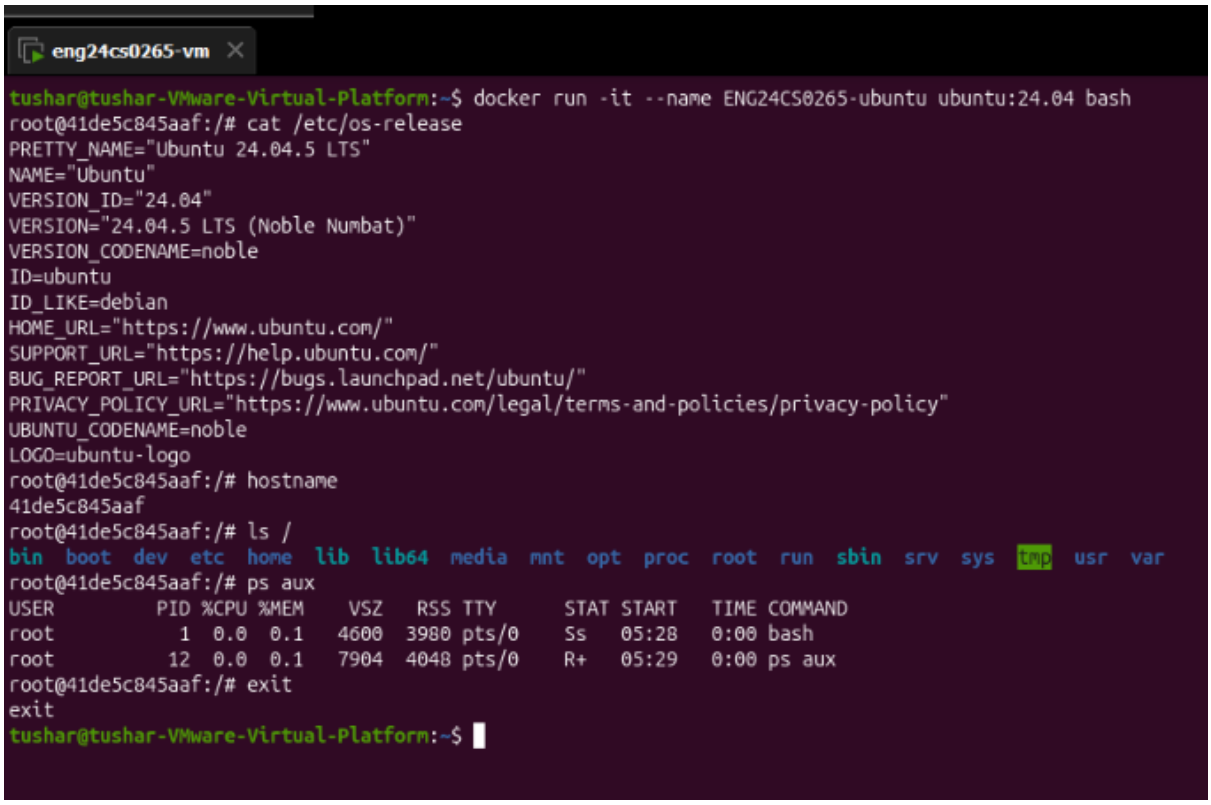
Answer: Alpine is so small because it is built on a minimal userland (musl libc and BusyBox) with no extra tools, while the Ubuntu image ships a fuller GNU userland with bash, apt, coreutils and more. My alpine image takes 13 MB on disk against 119 MB for ubuntu:24.04, roughly nine times less. Compared with the 25 GB virtual disk of the VM, the ubuntu image uses only about 0.5 % (119 MB of 25,600 MB), so many images fit easily on one VM.

Experiment A5 - Interactive container

Aim: Go inside a container and see that it is an isolated Linux environment.

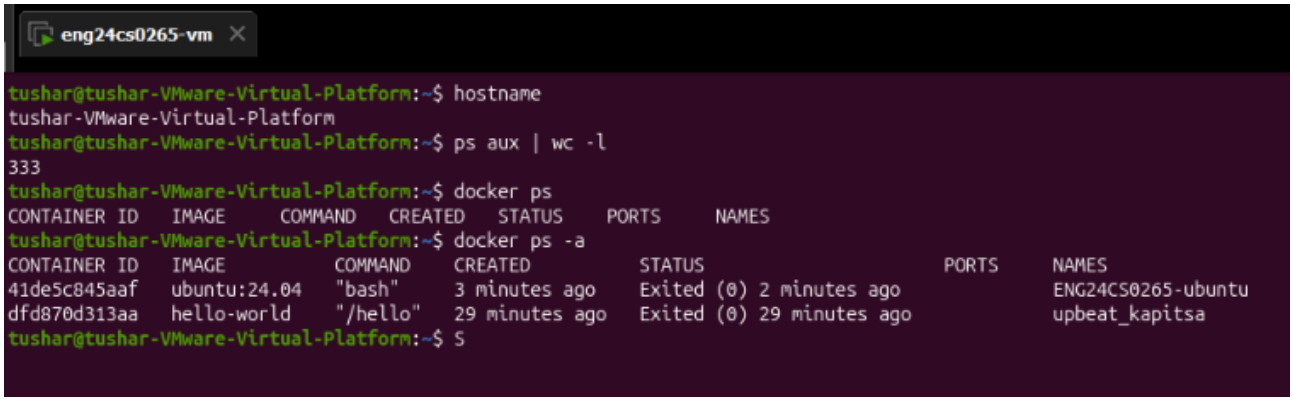
```
docker run -it --name ENG24CS0265-ubuntu ubuntu:24.04 bash
# inside: cat /etc/os-release ; hostname ; ls / ; ps aux ; exit
# back in the VM: hostname ; ps aux | wc -l ; docker ps ; docker ps -a
```

SS A5.1



Inside the container (prompt root@41de5c845aaf): os-release shows Ubuntu 24.04.5 LTS, the hostname is 41de5c845aaf, ls / lists a normal Linux root, and ps aux shows only bash and ps.

SS A5.2



Back on the VM: hostname, ps aux | wc -l (333 lines), an empty docker ps, and docker ps -a listing ENG24CS0265-ubuntu with status Exited (0), plus the earlier hello-world container.

Write: Compare the hostname and number of processes inside the container with those of the VM. Why does docker ps not show the container but docker ps -a does?

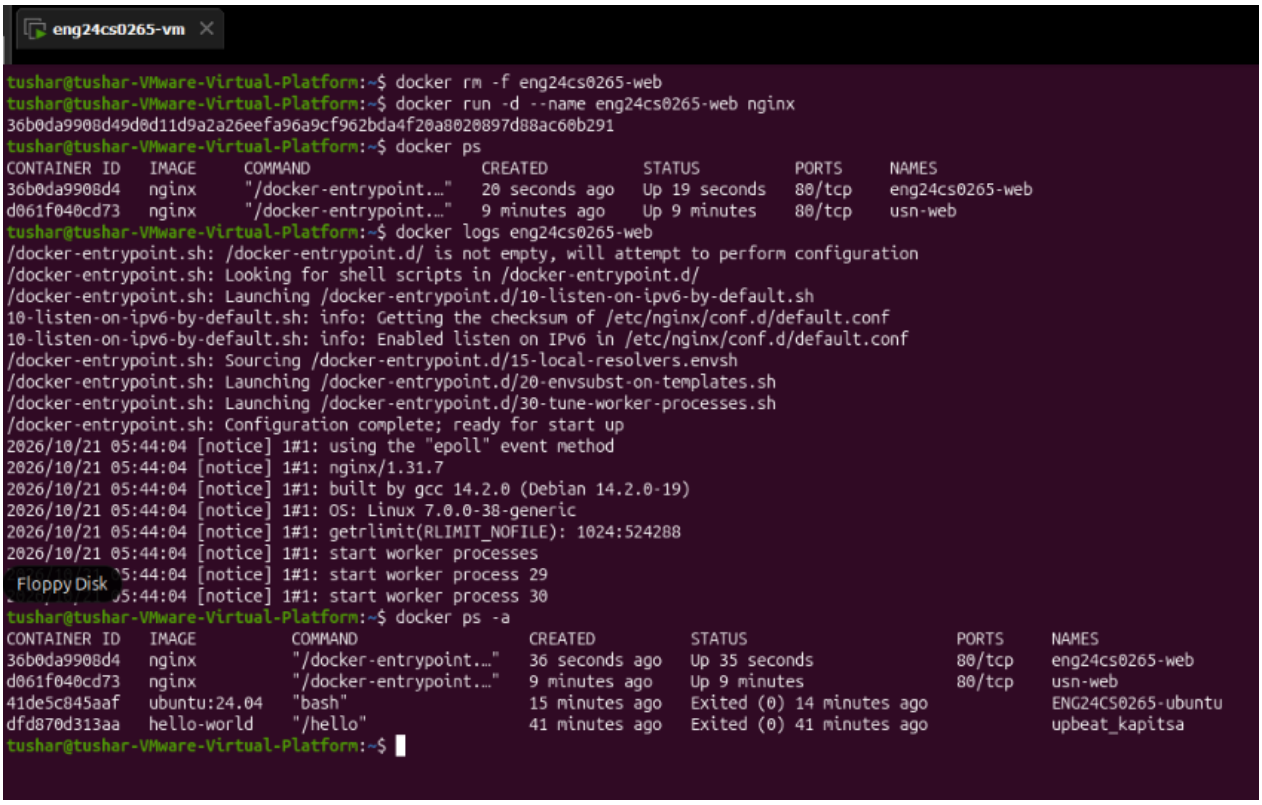
Answer: Inside the container the hostname is 41de5c845aaf (its container ID) and only 2 processes exist (bash as PID 1 and the ps command itself). The VM has the hostname tushar-VMware-Virtual-Platform and 333 lines of ps output (about 332 processes plus the header). The container cannot see the VM's processes because it has its own PID and UTS namespaces. docker ps shows only running containers; when I typed exit the main process (bash) ended, so the container stopped, and only docker ps -a, which lists stopped containers too, still shows it.

Experiment A6 - Container lifecycle

Aim: Run a container in the background and control it.

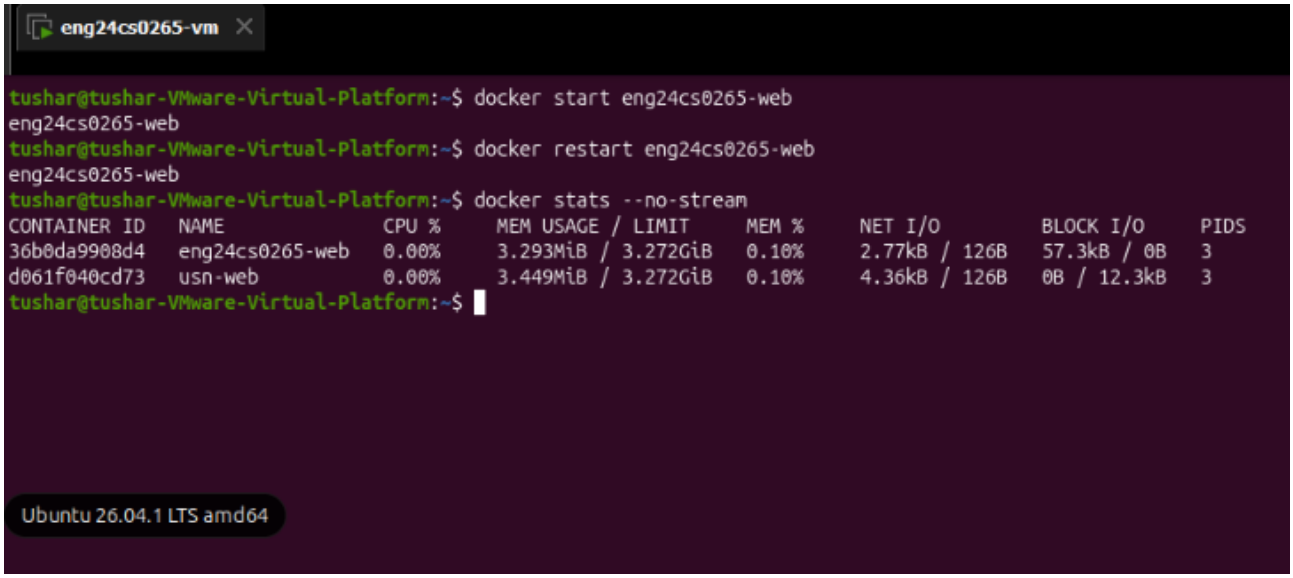
```
docker run -d --name eng24cs0265-web nginx
docker ps ; docker logs eng24cs0265-web ; docker ps -a
docker start eng24cs0265-web ; docker restart eng24cs0265-web ; docker stats --no-stream
```

SS A6.1



Old container removed with docker rm -f, then docker run -d starts nginx in the background (container ID printed); docker ps, docker logs eng24cs0265-web and docker ps -a show the running container and its nginx start-up log.

SS A6.2



docker start and docker restart on eng24cs0265-web, then docker stats --no-stream: both nginx containers use 0.00 % CPU and about 3.3 MiB of the 3.272 GiB available memory (0.10 %).

Write: What does the -d flag do? Draw the lifecycle: created → running → stopped → removed.

Answer: The -d flag means "detached": the container runs in the background, Docker prints only its ID and my terminal stays free for other commands (without -d nginx would keep the terminal attached to its output).

Lifecycle of a container:

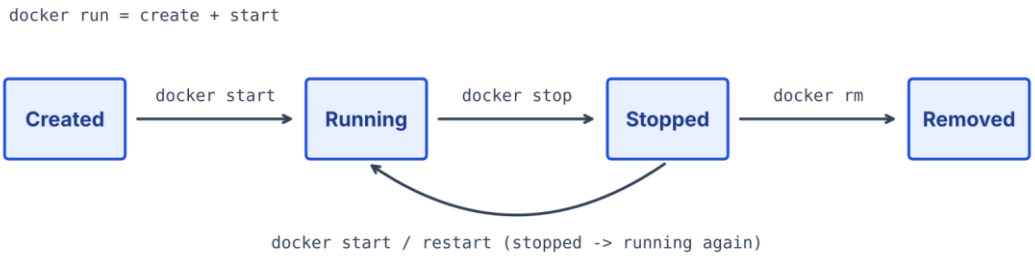


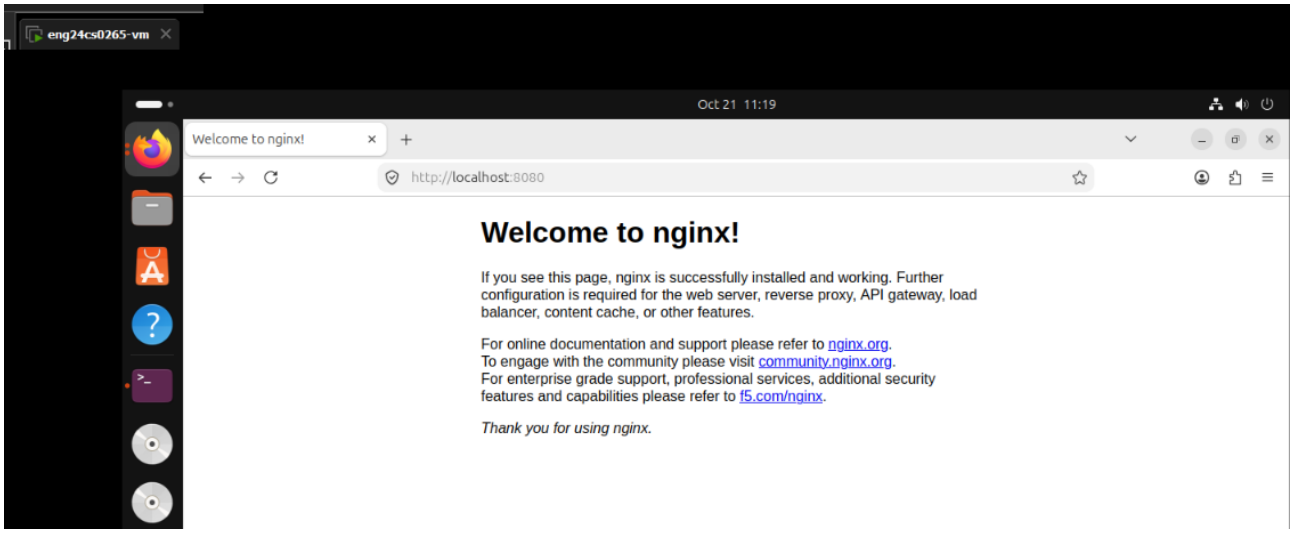
Figure: created → running → stopped → removed, with the Docker command that moves a container between states.

Experiment A7 - Port mapping: serve a web page

Aim: Access a web server running inside a container from a browser.

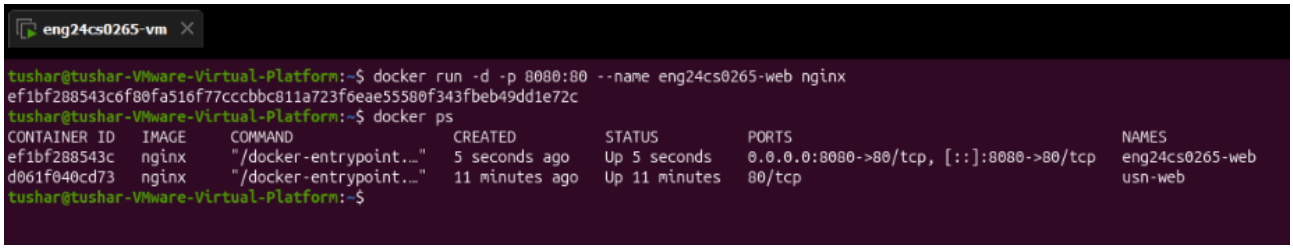
```
docker run -d -p 8080:80 --name eng24cs0265-web nginx
docker ps
# Firefox in the VM: http://localhost:8080
```

SS A7.1



Firefox inside the Ubuntu VM at http://localhost:8080 showing "Welcome to nginx!" served from the container.

SS A7.2



docker ps after starting the container: the PORTS column shows 0.0.0.0:8080->80/tcp (and [::]:8080->80/tcp) for eng24cs0265-web.

Write: Explain -p 8080:80 — which number is the VM's port and which is the container's port?

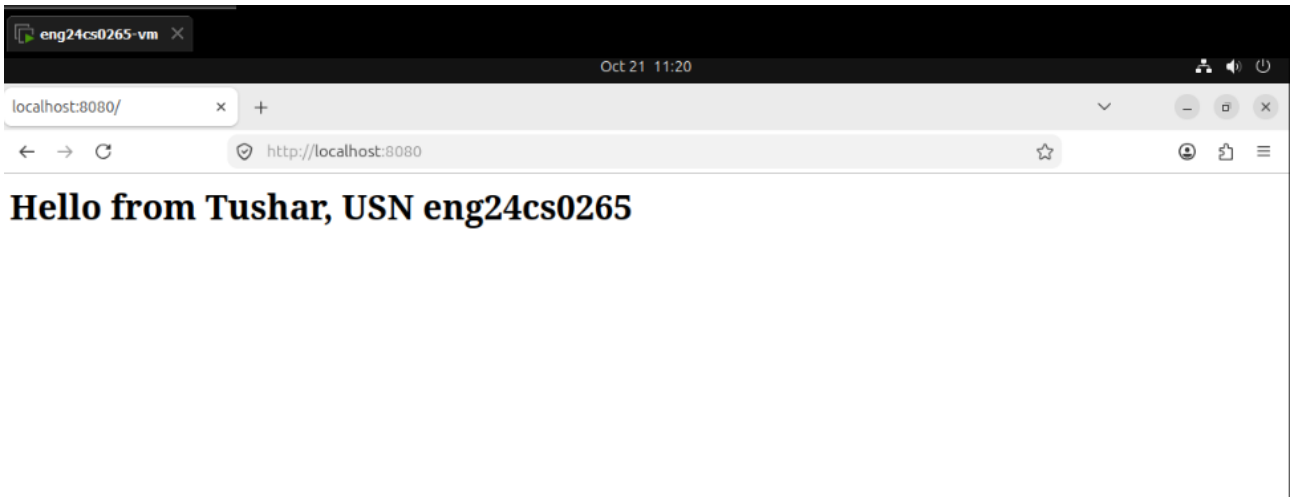
Answer: In -p 8080:80 the number before the colon (8080) is the port on the VM (the host) and the number after it (80) is the port inside the container where nginx listens. A request to localhost:8080 on the VM is forwarded to port 80 of the container.

Experiment A8 - Modify a running container

Aim: Execute commands inside a running container and see the change live.

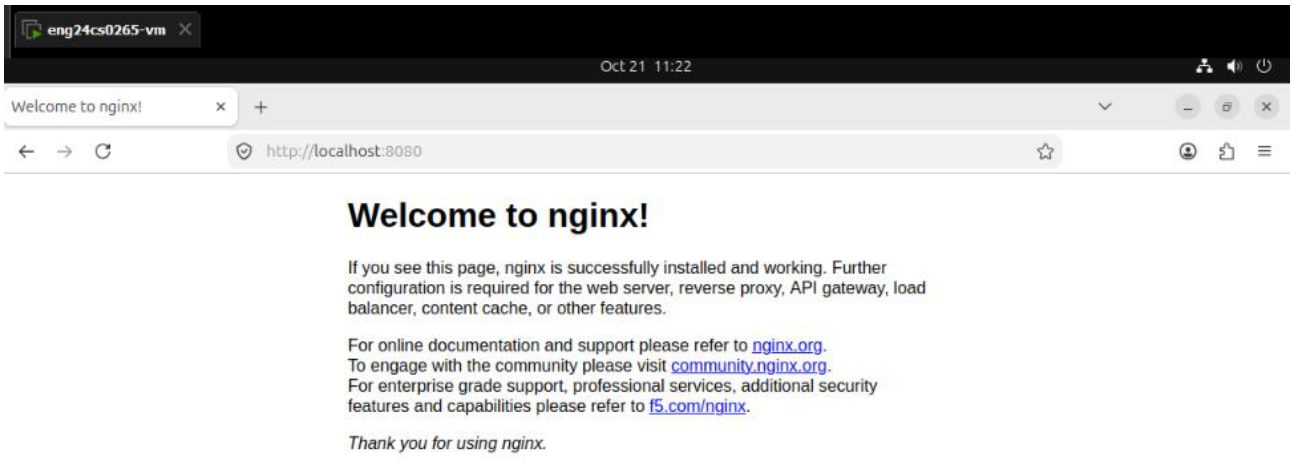
```
docker exec -it eng24cs0265-web sh
# echo "<h1>Hello from Tushar, USN eng24cs0265</h1>" > /usr/share/nginx/html/index.html
docker rm -f eng24cs0265-web
docker run -d -p 8080:80 --name eng24cs0265-web nginx
```

SS A8.1



After editing index.html inside the running container, localhost:8080 shows my custom page: "Hello from Tushar, USN eng24cs0265".

SS A8.2



After removing and recreating the container, localhost:8080 is back to the default "Welcome to nginx!" page.

Write: Your change disappeared. Why? What does this tell you about where container data lives?

Answer: My change disappeared because it was written only to the container's own writable layer, which is deleted together with the container. The new container started again from the unchanged nginx image. So data stored inside a container is temporary; to keep it, it has to live outside the container, in a volume (Experiment A10).

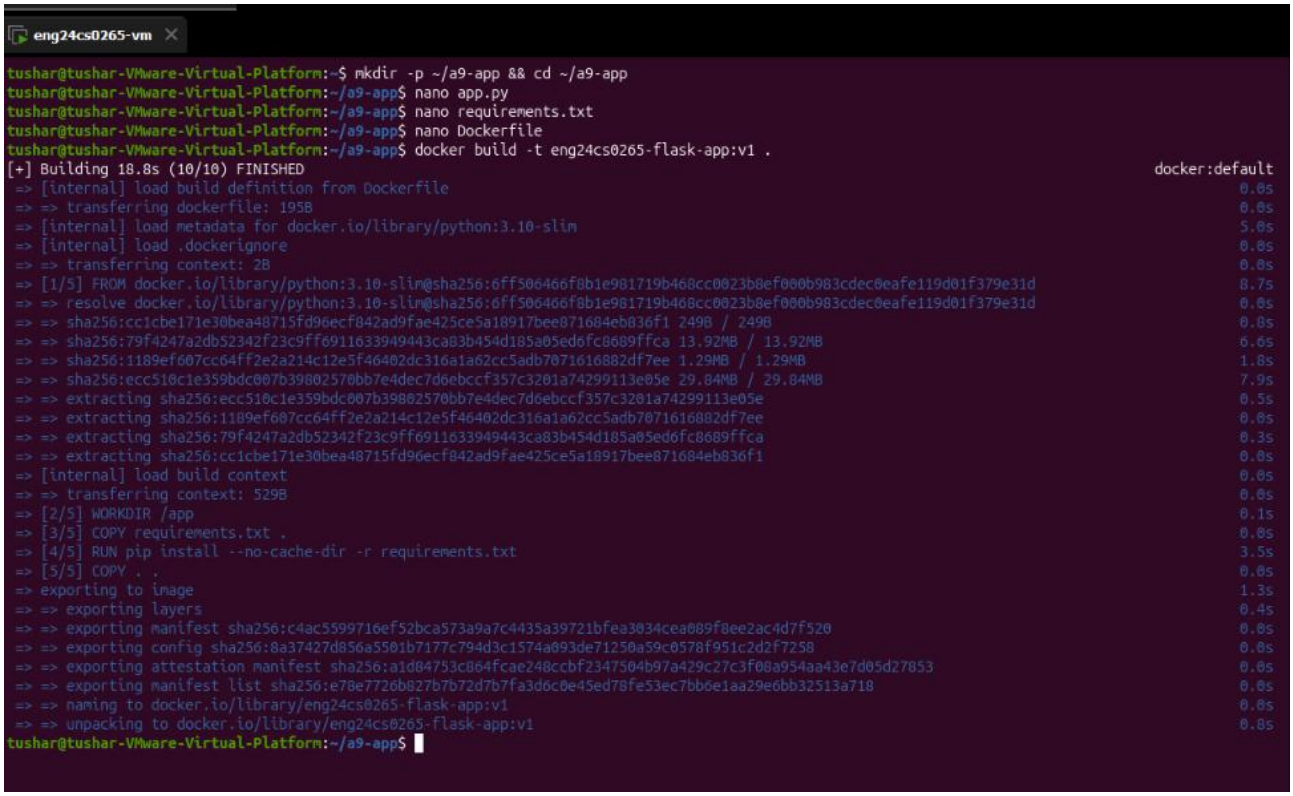
Experiment A9 - Build your own image with a Dockerfile

Aim: Package a small Python web app as a Docker image.

```
mkdir -p ~/a9-app && cd ~/a9-app        # app.py, requirements.txt, Dockerfile (written with nano)
docker build -t eng24cs0265-flask-app:v1 .
docker run -d -p 5000:5000 --name eng24cs0265-flask eng24cs0265-flask-app:v1
```

I packaged a small Flask application (app.py with requirements.txt) instead of the plain http.server example.

SS A9.1



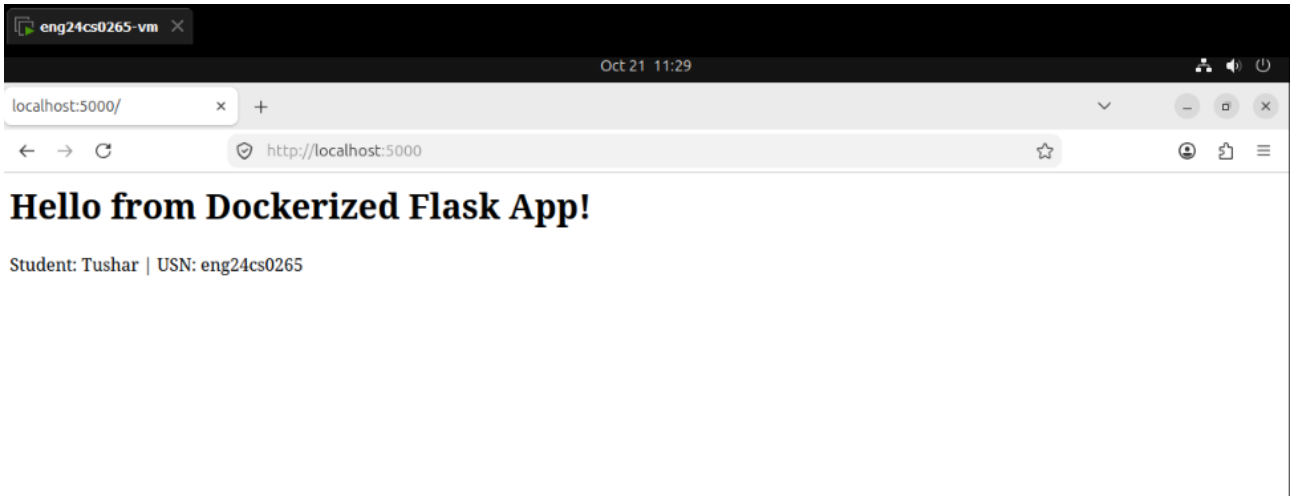
docker build -t eng24cs0265-flask-app:v1 . finishing in 18.8 s with the five build steps: FROM python:3.10-slim, WORKDIR /app, COPY requirements.txt, RUN pip install, COPY . . (image named eng24cs0265-flask-app:v1).

SS A9.2



docker ps showing the new container eng24cs0265-flask (image eng24cs0265-flask-app:v1, command "python app.py", port 0.0.0.0:5000->5000/tcp) running next to the nginx containers.

SS A9.3



Firefox inside the VM at localhost:5000 showing "Hello from Dockerized Flask App!" with "Student: Tushar | USN: eng24cs0265".

Write: Explain each of the 5 Dockerfile lines in one sentence. Compare the hostname in the browser with the container ID in docker ps .

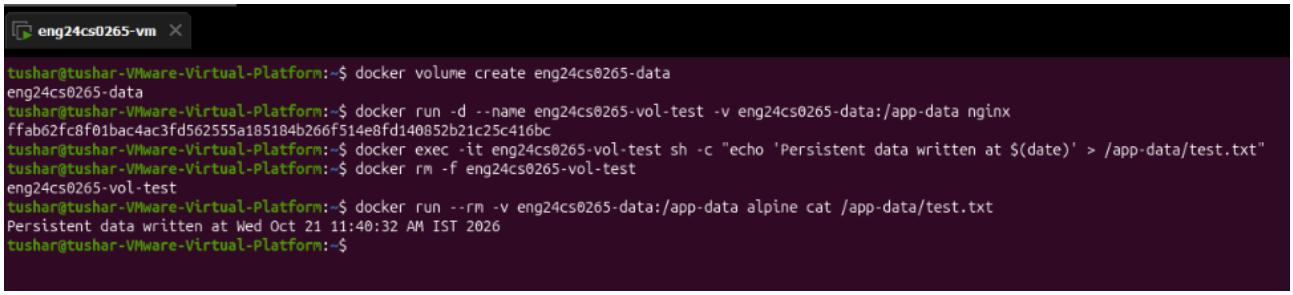
Answer: The build steps map to these Dockerfile instructions: FROM python:3.10-slim picks the small Python base image to start from; WORKDIR /app creates and switches to the /app folder inside the image; COPY requirements.txt . copies the dependency list first so Docker can cache that layer; RUN pip install --no-cache-dir -r requirements.txt installs Flask while the image is built; COPY . . copies the application code into the image. The container then starts the app with python app.py (the COMMAND column in docker ps) and listens on port 5000. A container's hostname is by default its short container ID, so the hostname inside eng24cs0265-flask is the ID shown by docker ps (75cda934e9f1); my page prints the student name and USN rather than the hostname.

Experiment A10 - Persist data with volumes

Aim: Store data that survives container deletion.

```
docker volume create eng24cs0265-data
docker run -d --name eng24cs0265-vol-test -v eng24cs0265-data:/app-data nginx
docker exec -it eng24cs0265-vol-test sh -c "echo 'Persistent data written at $(date)' > /app-data/test.txt"
docker rm -f eng24cs0265-vol-test
docker run --rm -v eng24cs0265-data:/app-data alpine cat /app-data/test.txt
```

SS A10.1



Volume eng24cs0265-data is created and mounted into a container, a file is written, the container is deleted, and a brand-new alpine container still reads "Persistent data written at Wed Oct 21 11:40:32 AM IST 2026".

Write: Contrast this with Experiment A8. Give one real-world case where a volume is essential.

Answer: In Experiment A8 the edited page was lost when the container was removed. Here the data lives in the named volume eng24cs0265-data, which is managed by Docker outside any container, so it survived the deletion of eng24cs0265-vol-test and was read by a completely different container. A volume is essential for a database such as MySQL or PostgreSQL: its data directory must stay intact when the database container is upgraded or replaced.

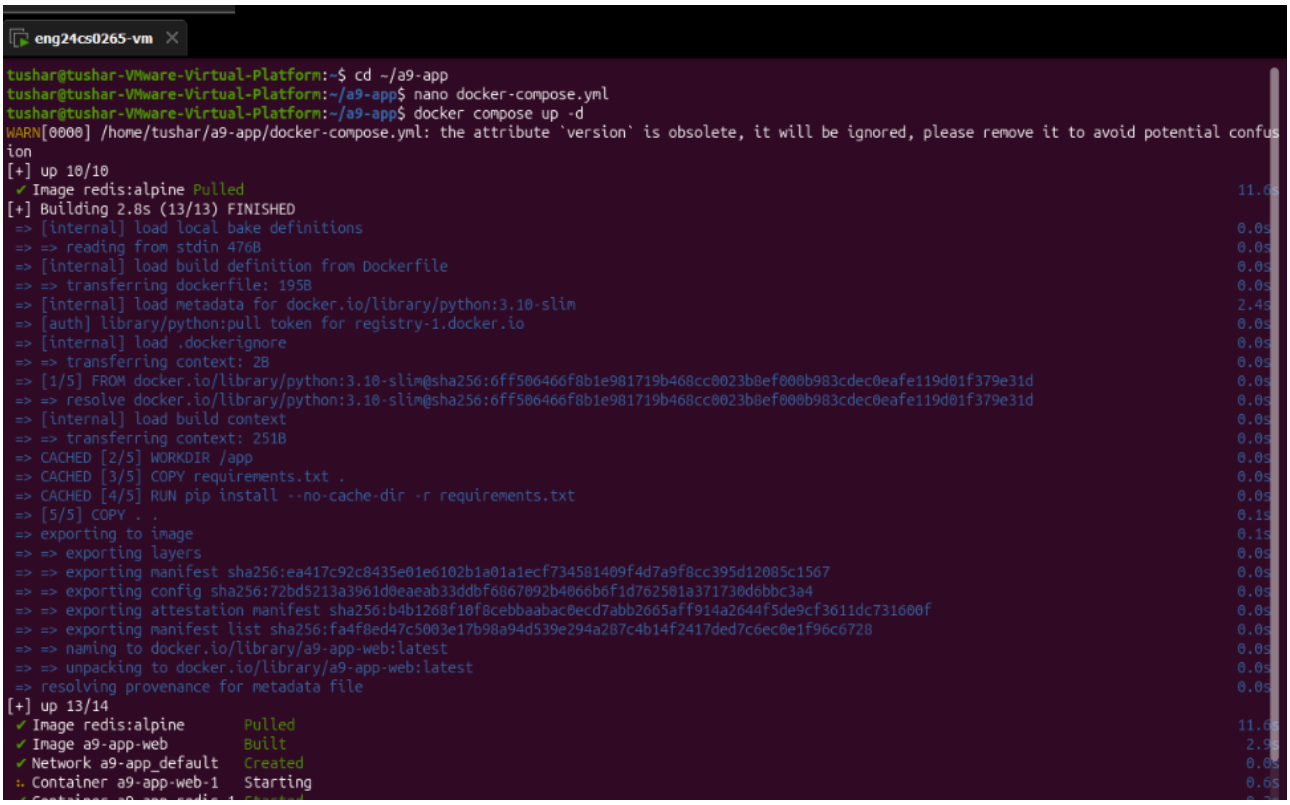
Experiment A11 - Multi-container app with Docker Compose, then clean up

Aim: Start several containers with one command, then free disk space.

```
docker compose up -d
docker compose ps
docker compose down
```

My compose file (docker-compose.yml in ~/a9-app) defines a web service built from the Flask Dockerfile and a cache service using redis:alpine.

SS A11.1



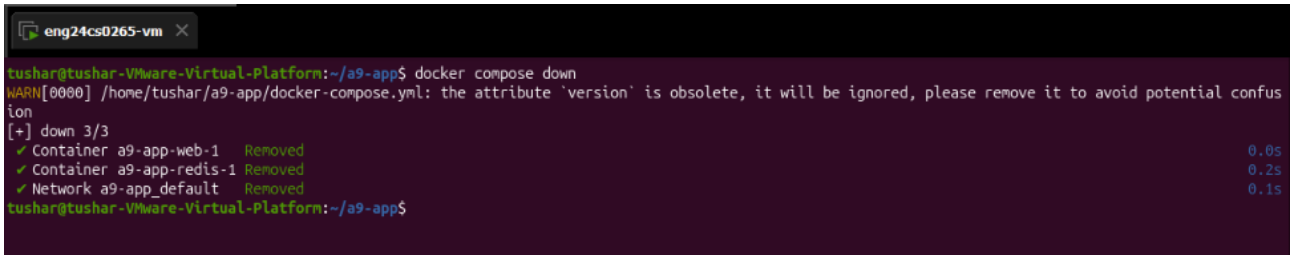
docker compose up -d: redis:alpine is pulled, the a9-app-web image is built, the network a9-app_default is created, and containers a9-app-web-1 and a9-app-redis-1 are started with one command.

SS A11.2



docker compose ps lists the a9-app-redis-1 container (redis:alpine, port 6379/tcp) as Up. Docker prints a warning that the "version" attribute in the compose file is obsolete and ignored.

SS A11.3



docker compose down stops and removes both containers (a9-app-web-1, a9-app-redis-1) and the a9-app_default network.

Write: What advantage does Compose give over running each container with docker run ?

Answer: Compose describes every service, its image, ports and settings in a single YAML file, so one command (docker compose up -d) starts the whole application and one command (down) removes it. It creates a shared network automatically so services can reach each other by service name, the setup is reproducible and can be kept in version control, and I do not have to type and remember several long docker run commands with all their flags.

Part B - Docker Desktop application

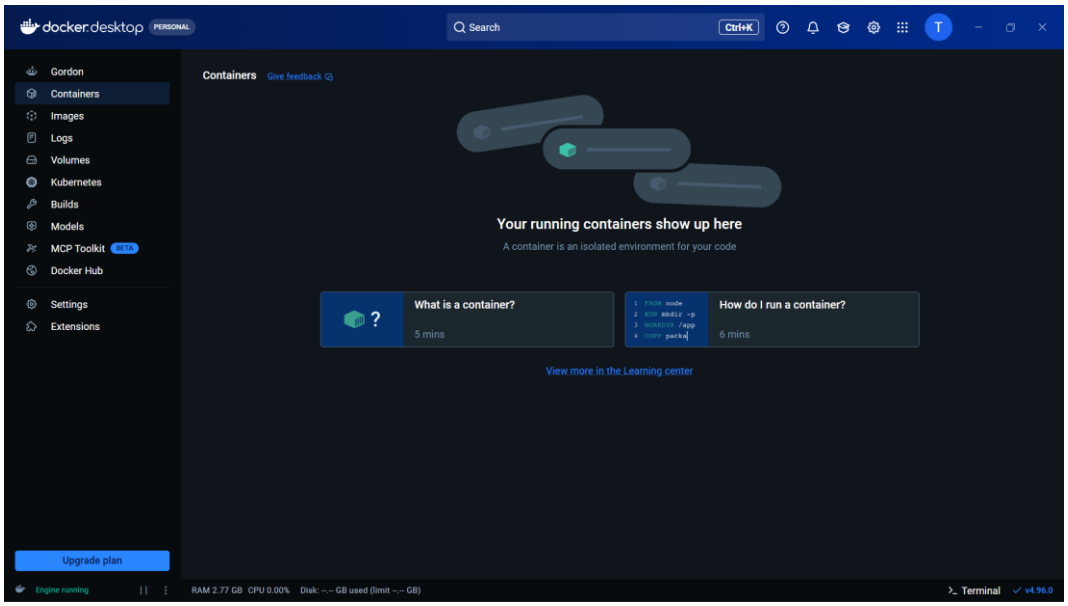
Part B was done on my own Windows laptop (the host, not the VM) with Docker Desktop 4.96.0 and the WSL 2 backend.

Experiment B1 - Install Docker Desktop and verify

Aim: Install the Docker Desktop application and confirm the engine is running.

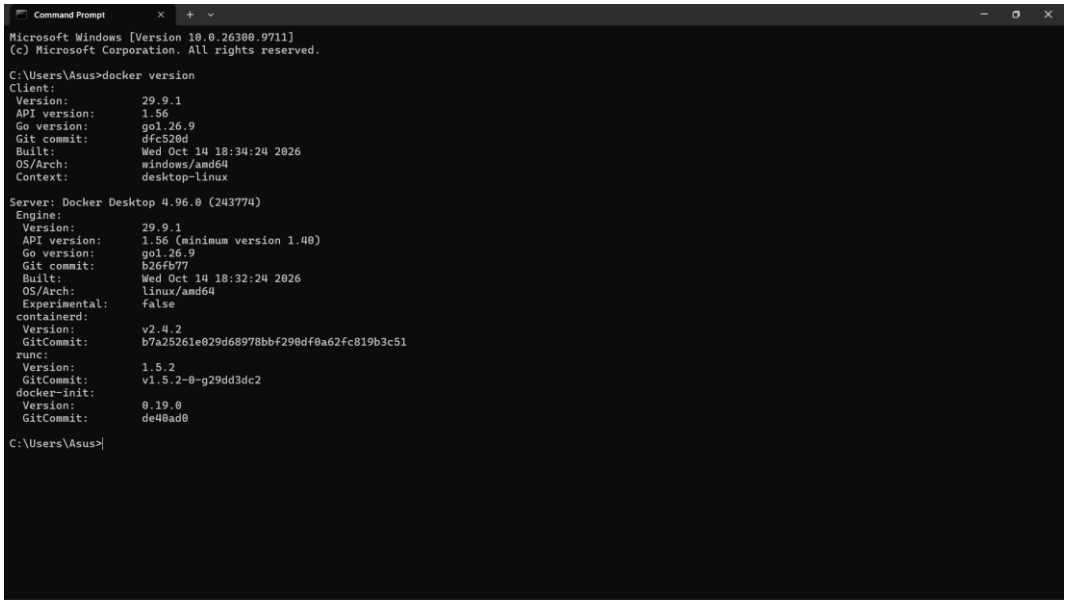
```
docker version
docker context ls
```

SS B1.1



Docker Desktop dashboard (Containers page) with "Engine running" in the bottom-left status bar and version v4.96.0 at the bottom right.

SS B1.2

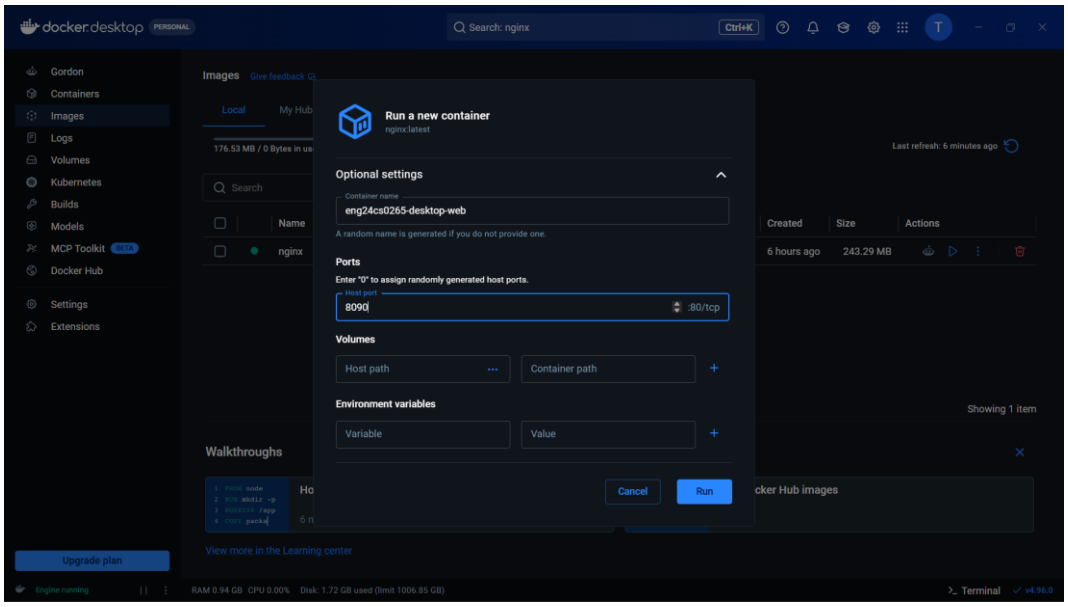


Command Prompt on the laptop: docker version prints the Client (windows/amd64, context desktop-linux) and "Server: Docker Desktop 4.96.0" with Engine 29.9.1 on linux/amd64.

Experiment B2 - Pull and run a container from the GUI

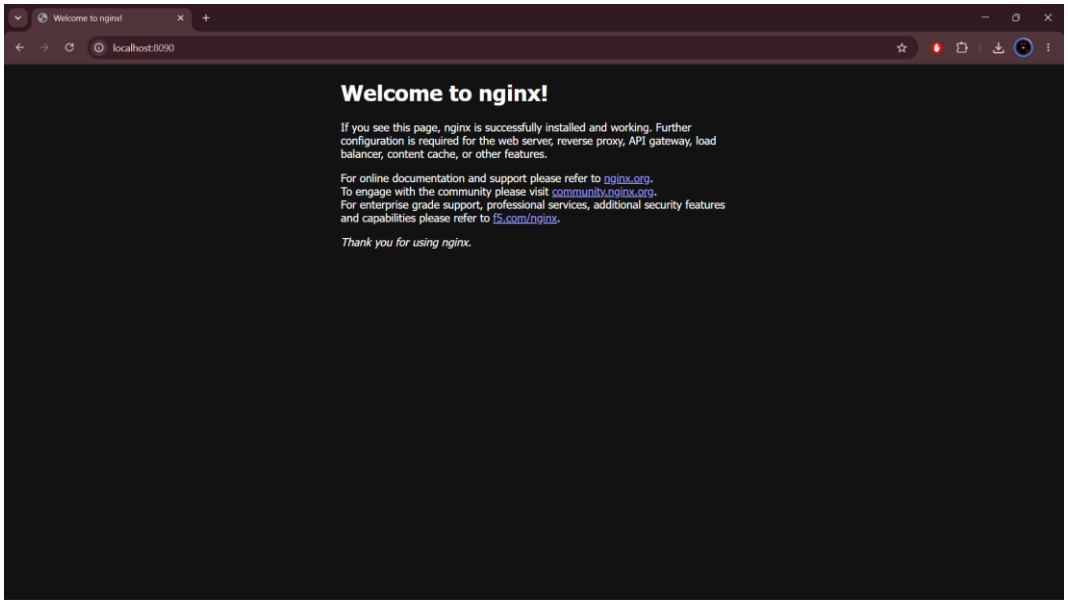
Aim: Run a web server without typing any docker run command.

SS B2.1



Docker Desktop "Run a new container" dialog for nginx:latest with Optional settings filled in: container name eng24cs0265-desktop-web and host port 8090 (mapped to 80/tcp).

SS B2.2

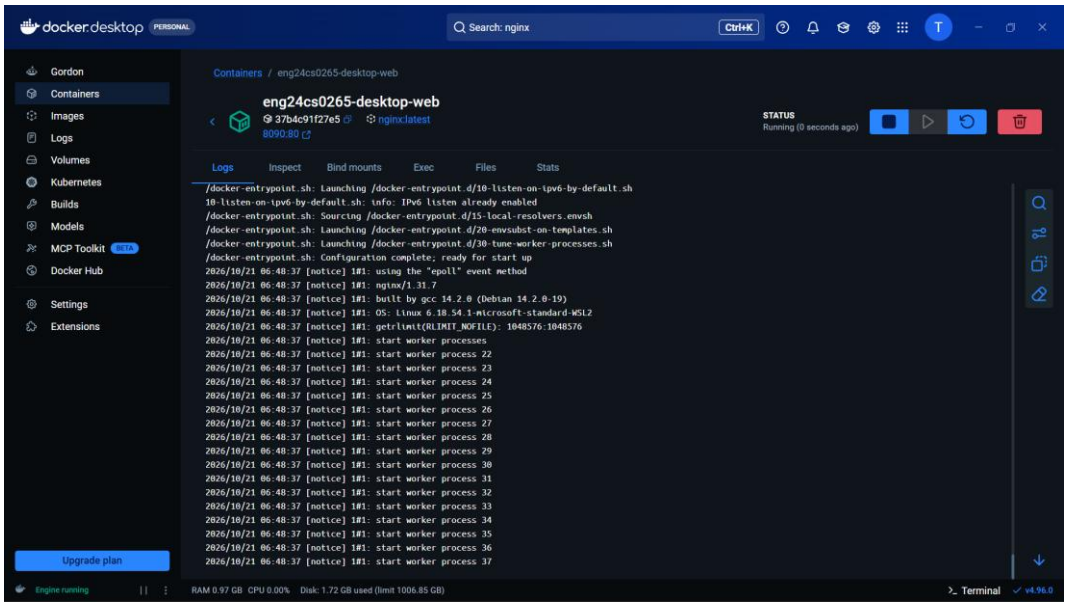


Browser on the laptop at localhost:8090 showing "Welcome to nginx!" from the container started in Docker Desktop.

Experiment B3 - Inspect and manage a running container

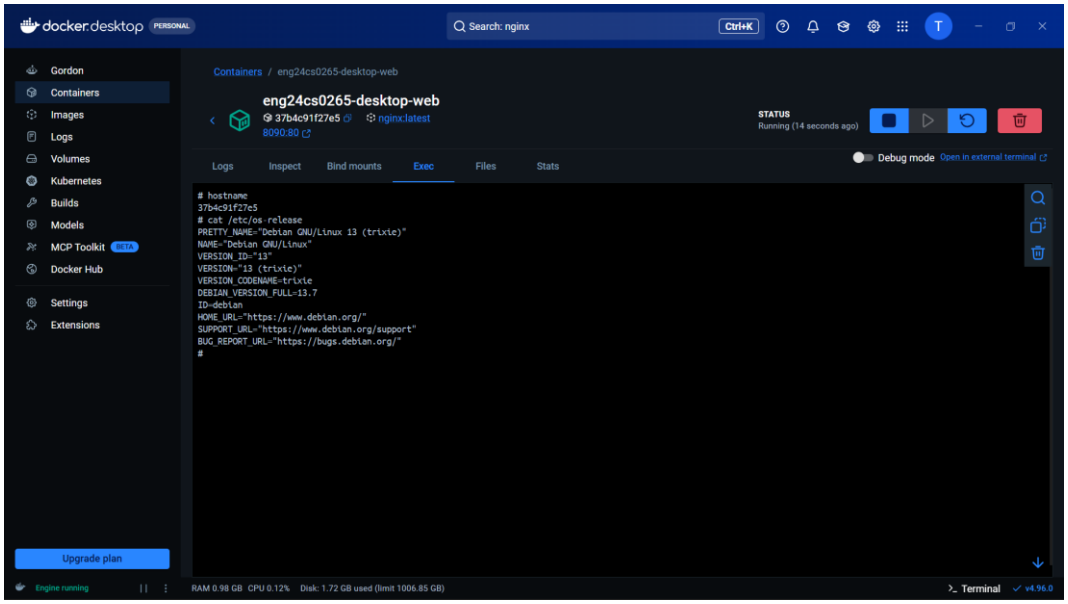
Aim: Use Docker Desktop's container tabs instead of CLI commands.

SS B3.1



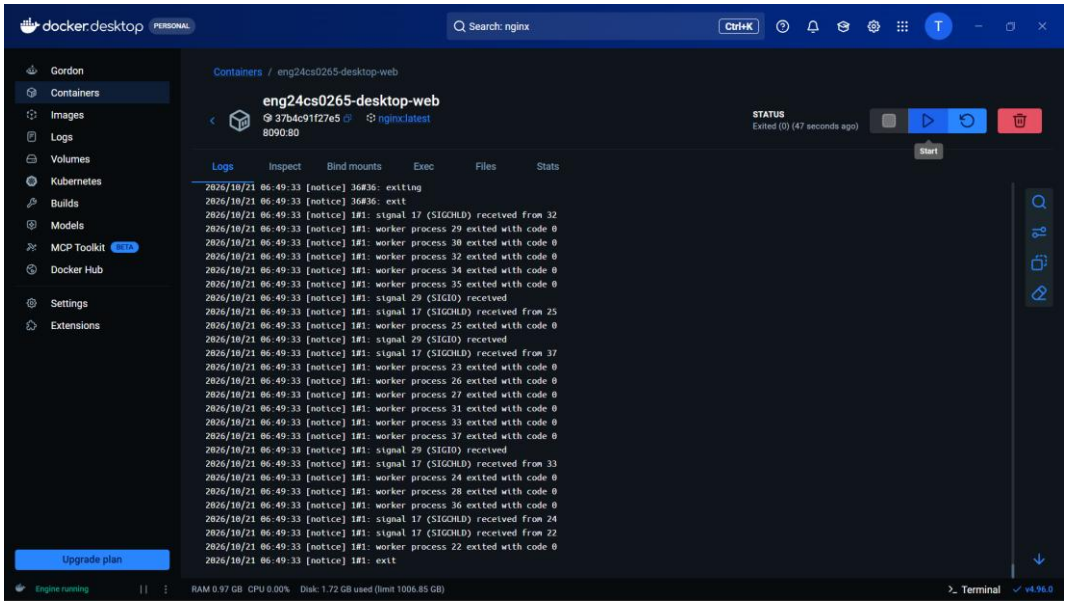
Logs tab of eng24cs0265-desktop-web (status Running, port 8090:80): the nginx start-up messages, including the kernel line "Linux 6.18.54.1-microsoft-standard-WSL2".

SS B3.2



Exec tab: hostname returns 37b4c91f27e5 (the container ID) and cat /etc/os-release shows the nginx image is based on Debian GNU/Linux 13 (trixie).

SS B3.3



After pressing Stop, the container eng24cs0265-desktop-web shows status "Exited (0)"; the Logs tab shows the nginx worker processes shutting down and the Start button is now active.

Write: For each tab (Logs, Inspect, Exec, Stats), write the equivalent CLI command you used in Part A.

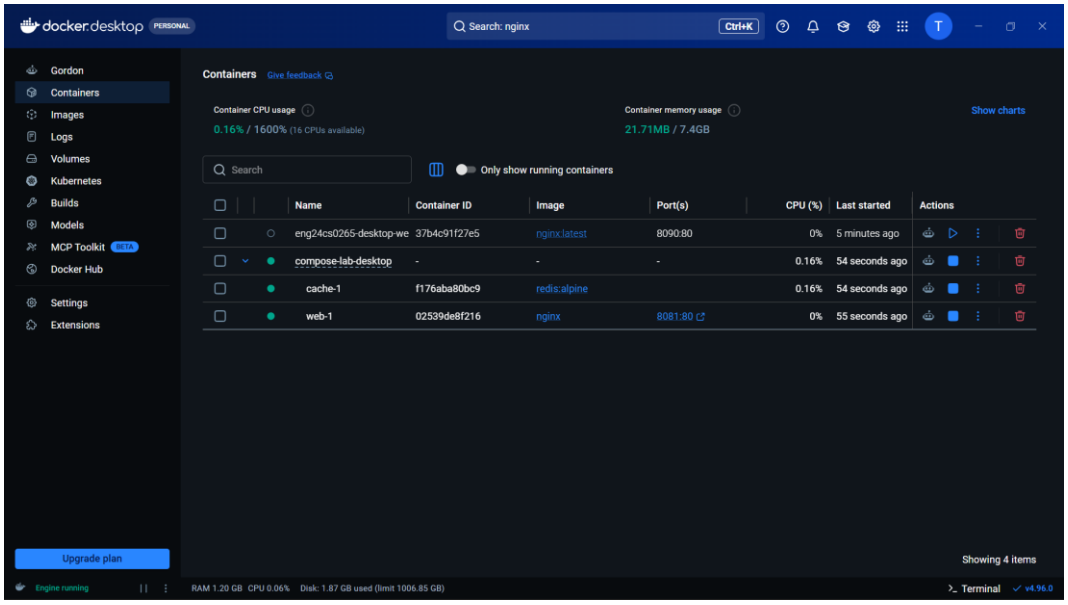
Answer: Equivalent CLI commands from Part A: Logs tab = docker logs eng24cs0265-web; Inspect tab = docker inspect <container> (IP address, port bindings; docker ps also lists the ports); Exec tab = docker exec -it eng24cs0265-web sh (then hostname, cat /etc/os-release); Stats tab = docker stats --no-stream. The Stop, Start and Delete buttons correspond to docker stop, docker start and docker rm.

Experiment B4 - Images, volumes and Compose stacks in the GUI

Aim: See Compose projects and volumes managed visually.

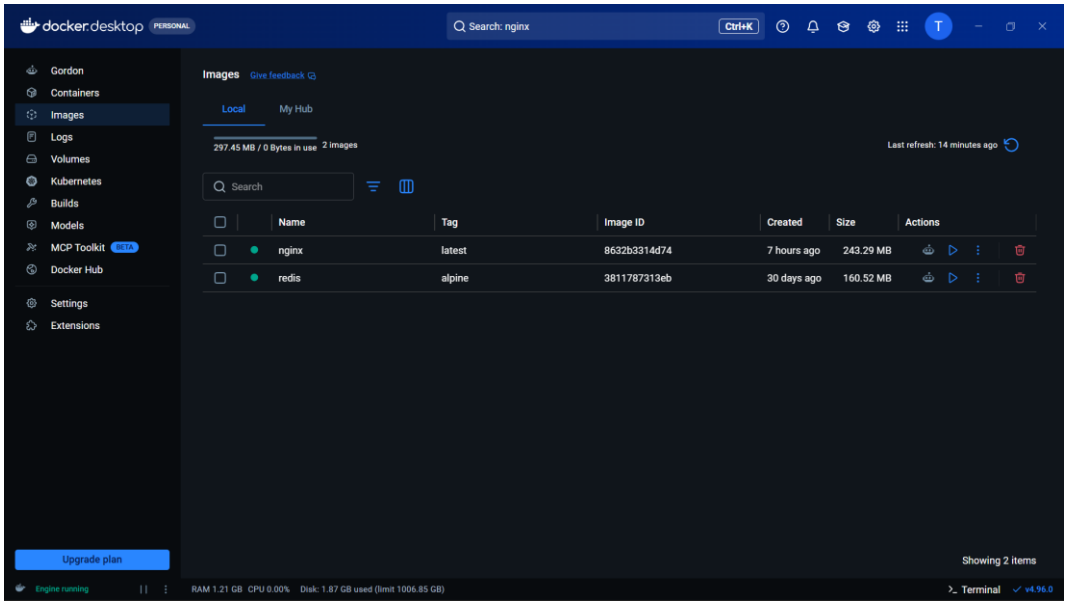
```
docker compose up -d
docker volume create eng24cs0265-desktop-data
```

SS B4.1



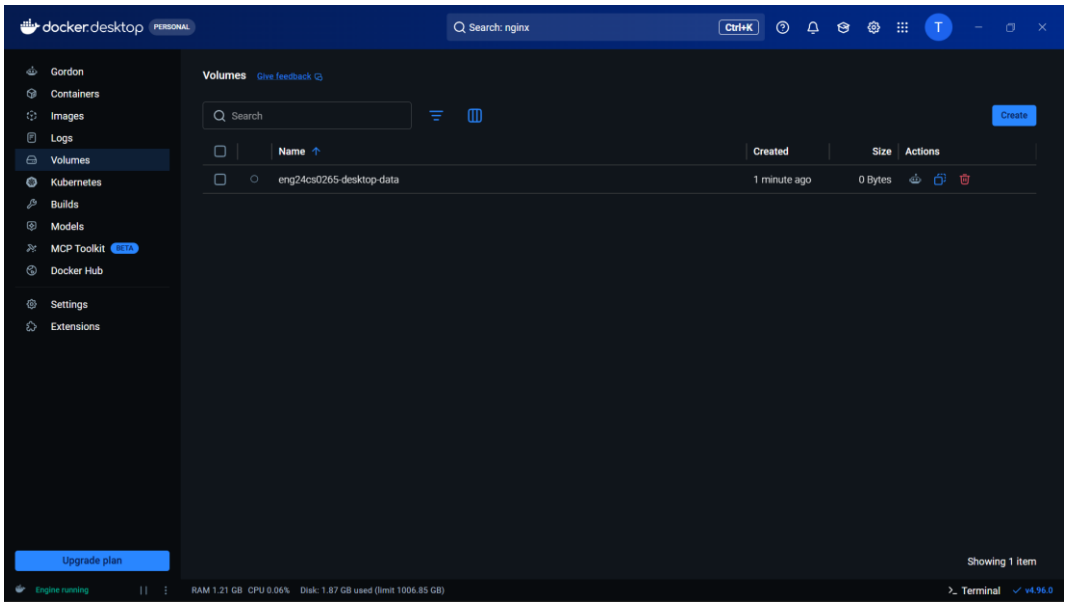
Containers tab with the Compose group compose-lab-desktop expanded: cache-1 (redis:alpine) and web-1 (nginx, port 8081:80) are both running; the earlier eng24cs0265-desktop-web is stopped.

SS B4.2



Images tab listing the pulled images with their sizes: nginx:latest 243.29 MB and redis:alpine 160.52 MB.

SS B4.3



Volumes tab showing the volume eng24cs0265-desktop-data (size 0 bytes) that I created.

Experiment B5 - Compare the two approaches

Aim: Reflect on VM-based Docker versus Docker Desktop.

Point of comparison	Part A: Docker Engine in VMware VM	Part B: Docker Desktop
Time taken to set up	45-60 mins – install VMware, create VM, install Ubuntu, then Docker Engine from the repository	5-10mins – install WSL 2 and Docker Desktop, wait for "Engine running"
Disk space used	25 GB virtual disk reserved for the VM (plus the Ubuntu ISO); Docker images inside are small (ubuntu 119 MB, alpine 13 MB)	About 1.7 - 2.1 GB of image/container data, as shown in the Docker Desktop status bar (1.72, 1.87 and 2.06 GB used)
RAM used while running	VM given 4 GB; inside it free -h showed 1.3 GiB used of 3.3 GiB, on top of the host's VMware memory	Docker Desktop status bar showed 0.94 - 1.21 GB RAM in use (2.77 GB while starting up)
How you ran containers	Terminal / Command Line (docker run, docker compose up)	Graphical User Interface (GUI click buttons in dashboard or host terminal)
How you viewed logs	CLI command (docker logs <container-name>)	GUI "Logs" tab directly inside the Docker Desktop dashboard
Which you would use for learning, and why	Helps you understand low-level Linux administration, permission models, system paths, and raw CLI commands essential for real-world DevOps roles.	Docker Desktop for quick visual feedback; it is easier for beginners to see images, volumes and logs
Which you would use on a production server, and why	Docker Engine on Linux: lightweight, headless, scriptable, no GUI overhead	Not on a production server: it is a desktop tool that adds a hidden VM and a GUI layer

Write: Docker Desktop on Windows and macOS also runs a small hidden Linux VM. Why does Docker need a Linux kernel, and how is this similar to what you did manually in Part A?

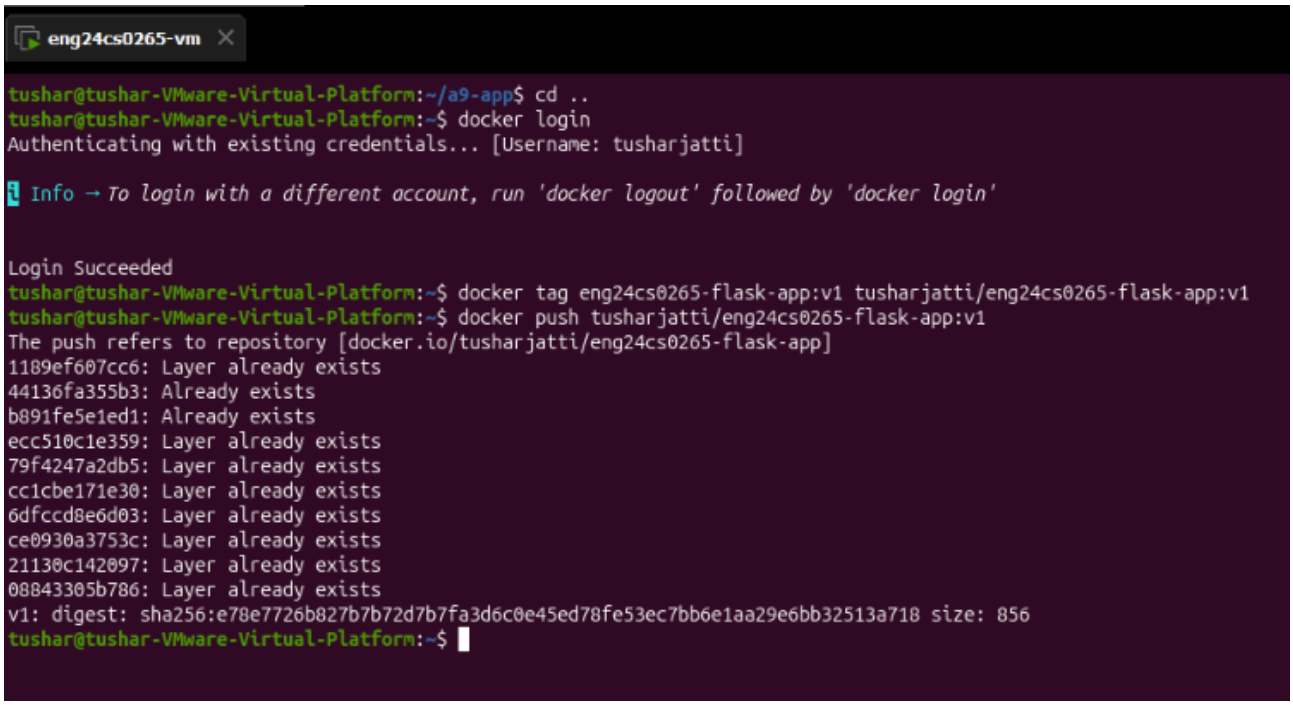
Answer: Containers are not a separate machine: they use features of the Linux kernel (namespaces for isolation and cgroups for resource limits), so a Linux kernel is required. Windows and macOS have different kernels, so Docker Desktop starts a small hidden Linux VM (WSL 2 on my laptop; the log in SS B3.1 shows a "microsoft-standard-WSL2" Linux kernel) and runs the Docker engine inside it. This is the same idea as Part A, where I manually created an Ubuntu VM in VMware and installed Docker Engine inside it to get a Linux kernel (the nginx log in SS A6.1 shows "Linux 7.0.0-38-generic"). Docker Desktop simply automates and hides that VM.

Bonus - Publish your image

Docker Hub account: tusharjatti. The Flask image from Experiment A9 was tagged as tusharjatti/eng24cs0265-flask-app:v1 and pushed from the VM, then pulled and run in Docker Desktop on the laptop.

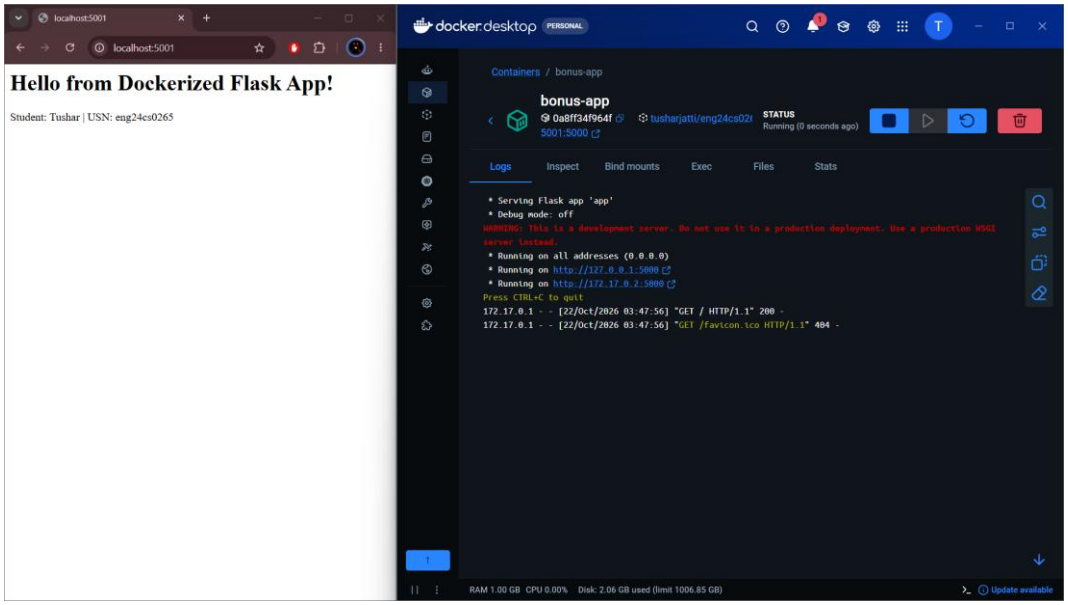
```
docker login
docker tag eng24cs0265-flask-app:v1 tusharjatti/eng24cs0265-flask-app:v1
docker push tusharjatti/eng24cs0265-flask-app:v1
```

SS Bonus.1



In the VM: docker login succeeds for user tusharjatti, the image is tagged and docker push uploads it to docker.io/tusharjatti/eng24cs0265-flask-app (layers listed, digest sha256:e78e7726...). In this run all layers already existed from my first push.

SS Bonus.2



On the laptop: Docker Desktop shows the container bonus-app running from tusharjatti/eng24cs0265-flask-app (port 5001:5000) with the Flask logs, and the browser at localhost:5001 shows "Student: Tushar | USN: eng24cs0265" - the same image works on both machines.

Command Summary

Command	Description
hostnamectl / lsb_release -a / free -h / nproc	Show host name, OS release, memory and CPU count of the VM
sudo apt update / apt install ...	Refresh package lists and install Docker packages
sudo usermod -aG docker $USER	Allow my user to run docker without sudo
docker --version / docker version / docker compose version	Show client, server and Compose versions
systemctl status docker	Check that the Docker service is active (running)
docker run hello-world	Pull (if needed) and run a test container
docker pull <image>	Download an image from Docker Hub
docker images	List local images with their sizes
docker history <image>	Show the layers an image is built from
docker run -it --name <n> <image> bash	Run an interactive container with a terminal
docker run -d --name <n> <image>	Run a container in the background (detached)
docker run -d -p 8080:80 ...	Run a container and map host port 8080 to container port 80
docker ps / docker ps -a	List running containers / all containers including stopped
docker logs <name>	Show a container's output
docker start / docker restart / docker stop <name>	Start, restart or stop a container
docker stats --no-stream	Show CPU and memory use once
docker exec -it <name> sh	Run a command or shell inside a running container
docker rm -f <name>	Force-remove a container
docker build -t <name:tag> .	Build an image from a Dockerfile
docker volume create <name>	Create a named volume
docker run --rm -v <vol>:/path ...	Run a temporary container with a volume mounted
docker compose up -d / ps / down	Start, list and remove the services of a compose file
docker login / tag / push	Log in to Docker Hub, tag an image and upload it
docker context ls	List Docker contexts (host check in Part B)

Conclusion

In this assignment I learned how containers differ from virtual machines and used both together.

In Part A, I created an Ubuntu VM in VMware, installed Docker Engine and worked through the whole lifecycle from the command line: pulling images, running interactive and background containers, mapping ports, building my own Flask image with a Dockerfile, keeping data in a volume and running several services with Docker Compose. I saw that data inside a container is temporary (Experiment A8) while a volume keeps it (A10), and that an image I push to Docker Hub runs the same way on a different machine (Bonus).

In Part B, Docker Desktop on my laptop gave the same results through a graphical interface and made logs, images and volumes easy to inspect, and I understood that it also relies on a hidden Linux VM (WSL 2).

A problem I faced: when I started a new nginx container with a name I had already used, Docker refused with a "container name is already in use" conflict, and a port can also be blocked by another running container. I fixed it by listing containers with docker ps -a, removing the old one with docker rm -f <name> and running the command again (visible at the start of SS A6.1), or by choosing a different host port.

DAYANANDA SAGAR UNIVERSITY

Department of Computer Science & Engineering

AI-Driven Software Engineering & Project Management

Course Code: 24CS3502 · Unit 3 - Continuous Integration / Continuous Delivery with Jenkins

Jenkins CI/CD - Hands-on Assignment

Experiments J1 - J13

Name	Tushar Jatti
USN	ENG24CS0265
Roll no.	61
Section	5D
Mode	Individual
Date of submission	31st October 2026 (31/10/26)

System Details

Item	Details
Virtual machine	eng24cs0265-vm (VMware Workstation) · 4 GB RAM · 2 CPU cores · 25 GB disk · NAT
Ubuntu version	Ubuntu 26.04.1 LTS, kernel Linux 7.0.0-38-generic (used instead of 24.04 LTS)
Java version	OpenJDK 21.0.12.1 (OpenJDK Runtime Environment, 64-Bit Server VM)
Jenkins version	Jenkins 2.580.1 (LTS), running as the service jenkins on port 8080
Docker version	Docker Engine - Community 29.9.1 (installed in the Docker assignment)
Python / pytest in the VM	Python 3.14.4, pytest 9.0.2 (as printed in the Jenkins console output)
Jenkins admin user	eng24cs0265 (full name TUSHAR); second user eng24cs0265-dev
GitHub repository URL	https://github.com/TUSHAR751370/eng24cs0265-jenkins-lab

All job names start with my USN in lowercase (for example eng24cs0265-hello-freestyle), the Jenkins admin username is my USN, and the GitHub repository is named eng24cs0265-jenkins-lab.

Background in brief

Continuous Integration means that every code push is built and tested automatically on a server, so a broken change is found within minutes. Continuous Delivery goes further: every change that passes the tests is packaged and ready to deploy, often after one approval click. Jenkins is a free, open-source automation server written in Java that runs such pipelines; almost every feature comes from a plugin. In this assignment Jenkins was installed in my Ubuntu VM and used to build freestyle jobs and a pipeline that pulls code from GitHub, tests and packages it.

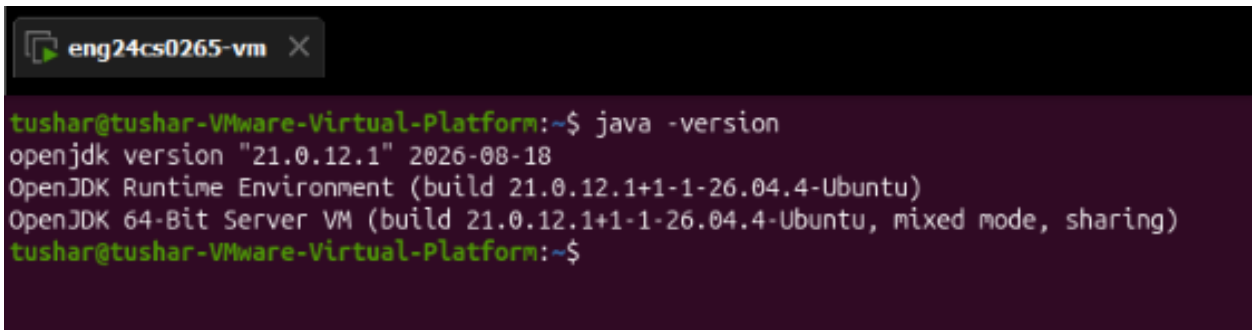
Part 1 - Install and explore Jenkins

Experiment J1 - Install Java and Jenkins in the Ubuntu VM

Aim: Install Jenkins as a service on Ubuntu.

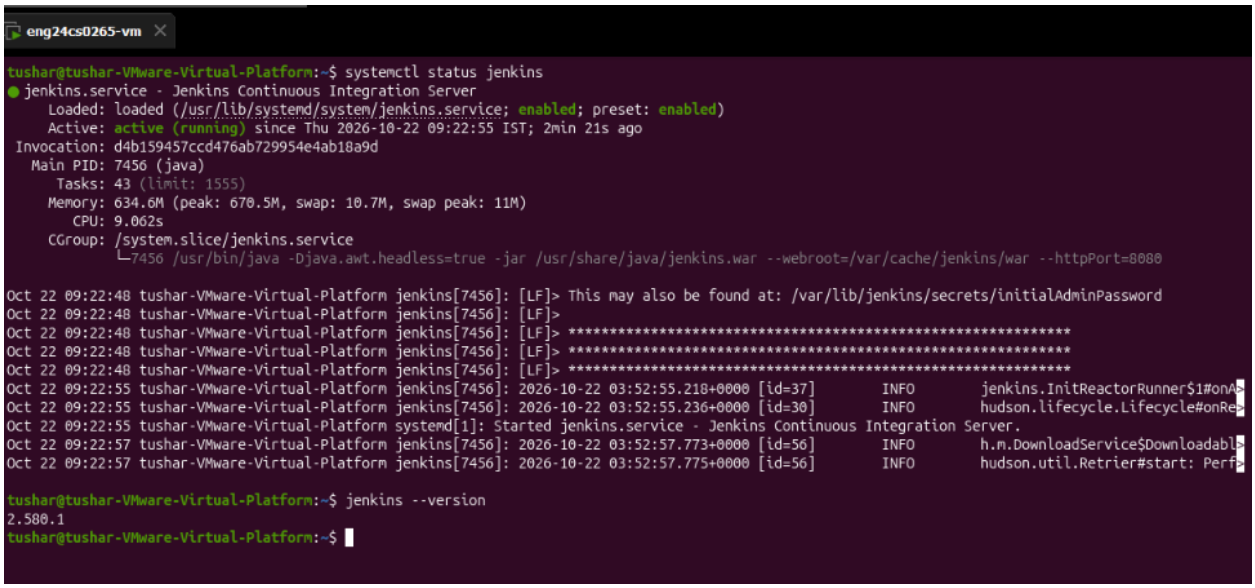
```
sudo apt install -y fontconfig openjdk-21-jre git python3-pytest
java -version
sudo apt install -y jenkins        # after adding the Jenkins LTS repository
sudo systemctl status jenkins
jenkins --version
```

SS J1.1



java -version in the VM terminal (prompt tushar@tushar-VMware-Virtual-Platform): OpenJDK version "21.0.12.1", so Java 21 is installed.

SS J1.2



systemctl status jenkins reports "active (running)" for jenkins.service, and jenkins --version prints 2.580.1.

Write: Jenkins runs as a Linux user called jenkins , and its home folder is /var/lib/jenkins . Run id jenkins and sudo ls /var/lib/jenkins and describe what you see.

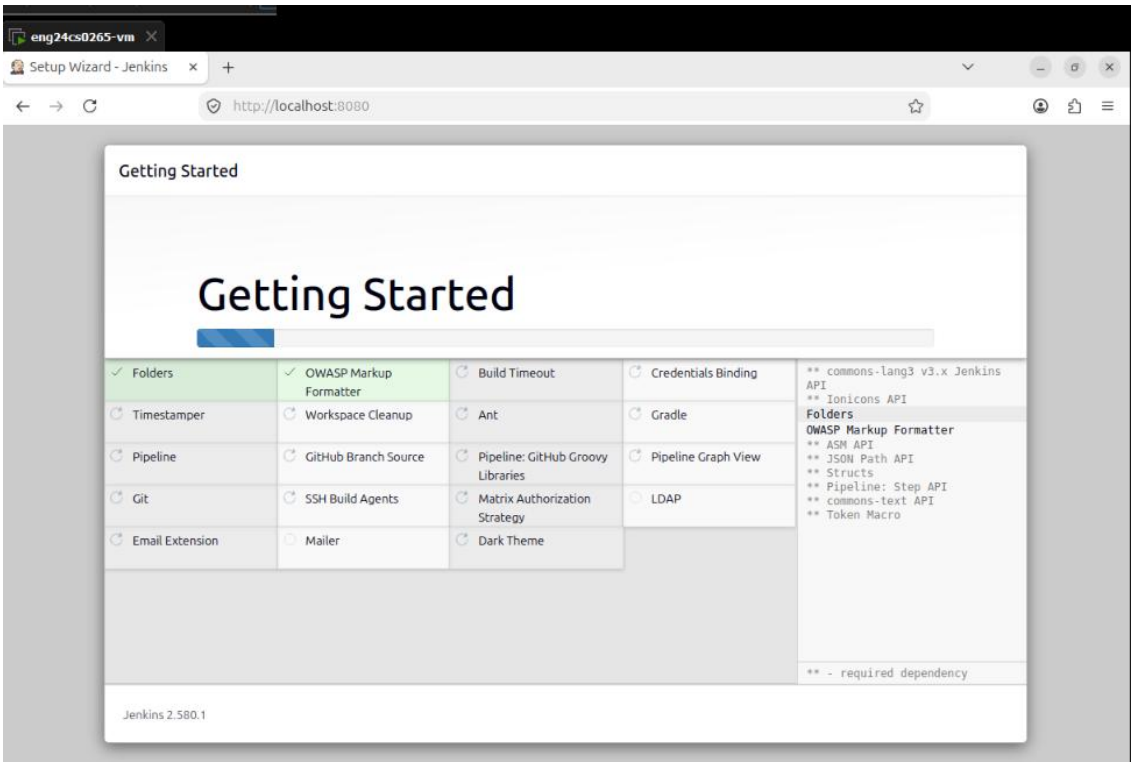
Answer: The Jenkins package creates a normal system account called jenkins, and the service runs as that user, not as root. Its home folder /var/lib/jenkins holds everything Jenkins keeps: configuration, the jobs folder (listed in SS J13.4), plugins, users, the secrets folder (the status log in SS J1.2 points to secrets/initialAdminPassword) and the workspace folder where jobs are built (SS J4.2). Files created by builds are owned by jenkins, as the ls -l output in SS J4.2 shows.

Experiment J2 - Unlock Jenkins and complete the setup wizard

Aim: Finish the first-time setup in the browser.

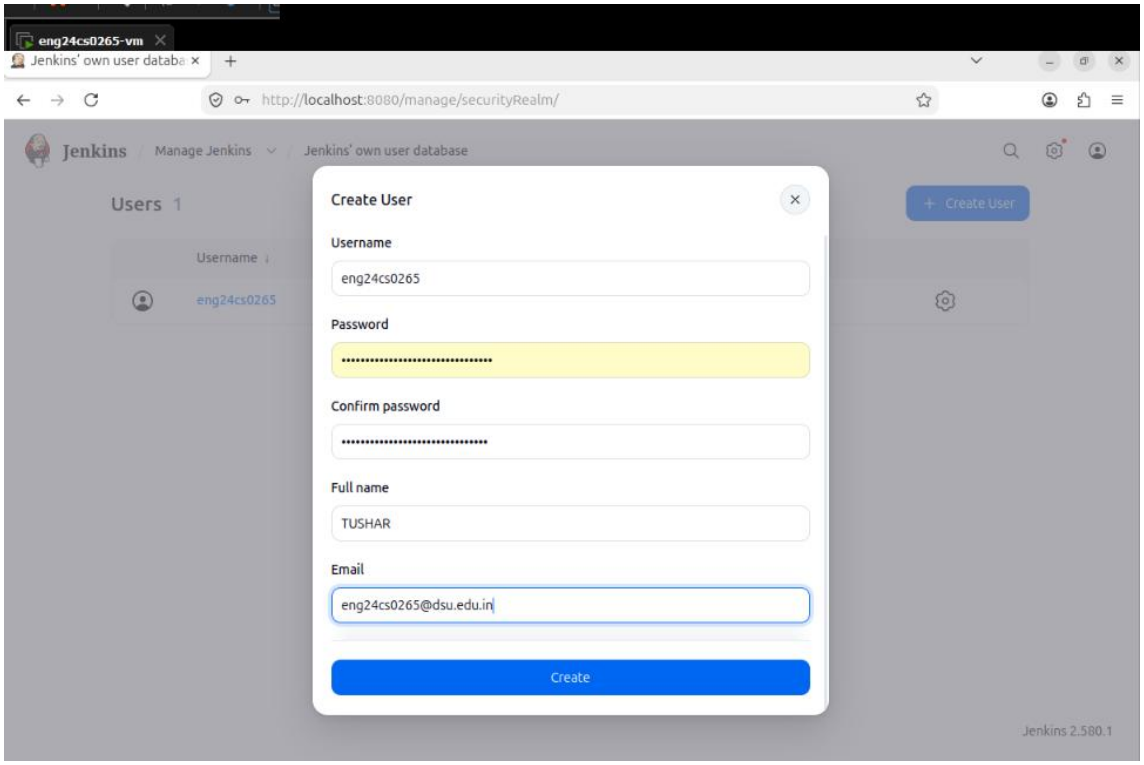
```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

SS J2.1



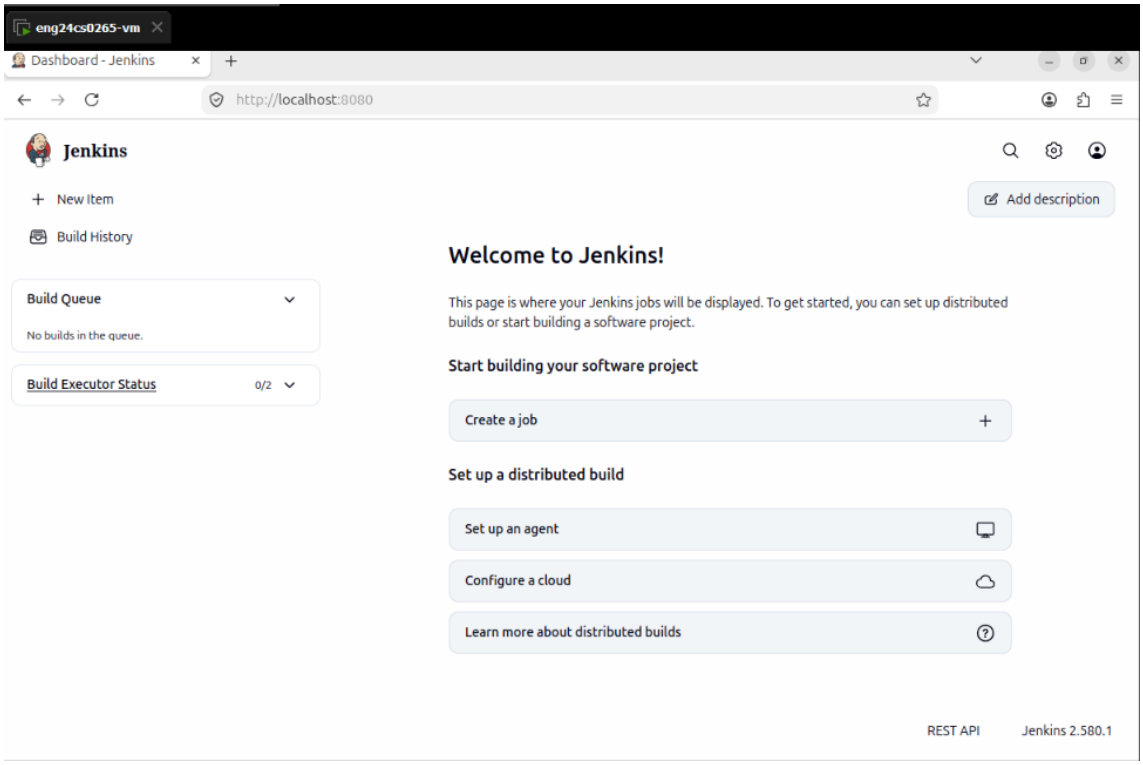
Firefox in the VM at localhost:8080 showing "Getting Started": the suggested plugins (Folders, Git, Pipeline, Timestamper and others) are being installed.

SS J2.2



Create User form for the admin account: username eng24cs0265 (my USN), full name TUSHAR and my college e-mail; the password fields are masked.

SS J2.3

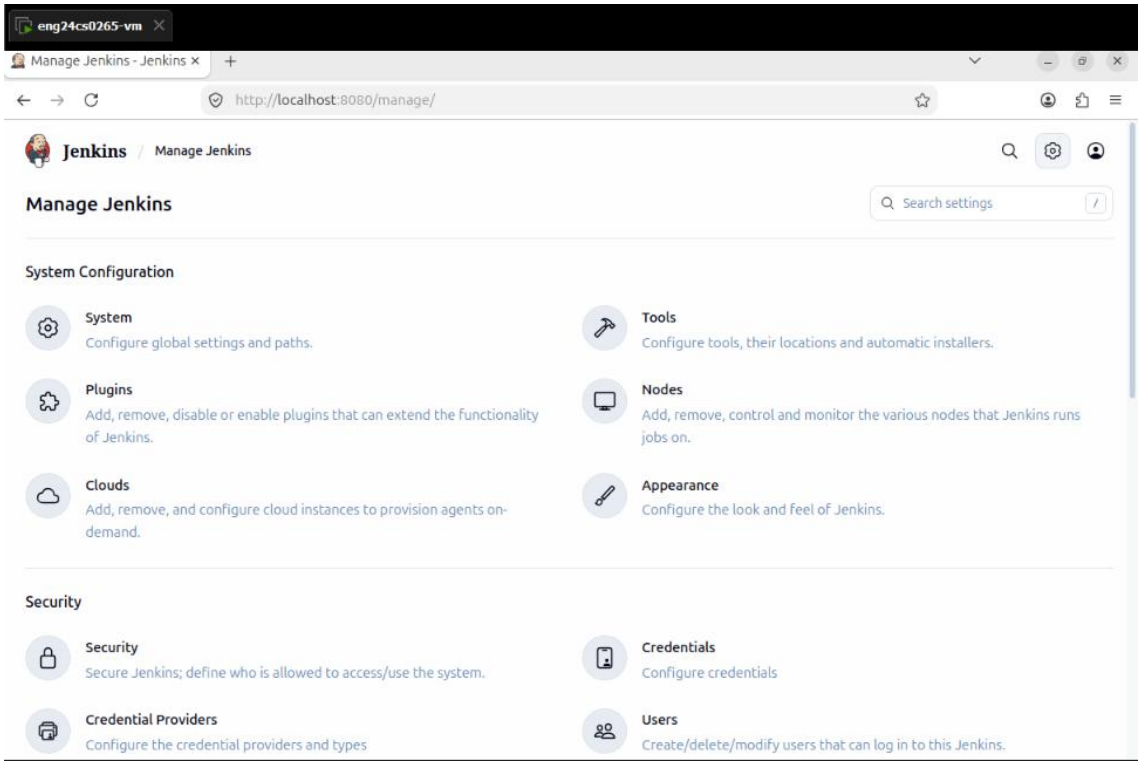


The Jenkins Dashboard ("Welcome to Jenkins!") after setup. The Build Executor Status panel shows 0/2, meaning the Built-In Node has two executors.

Experiment J3 - Tour of the Jenkins interface

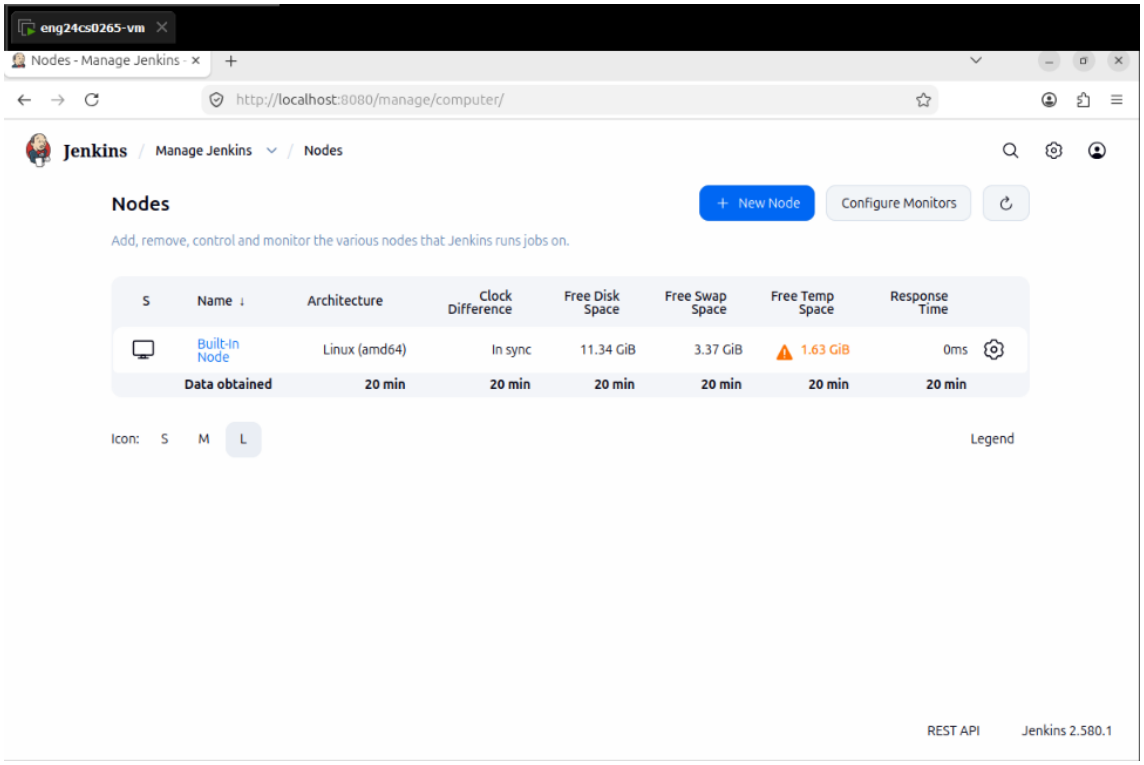
Aim: Learn where things are before creating jobs.

SS J3.1



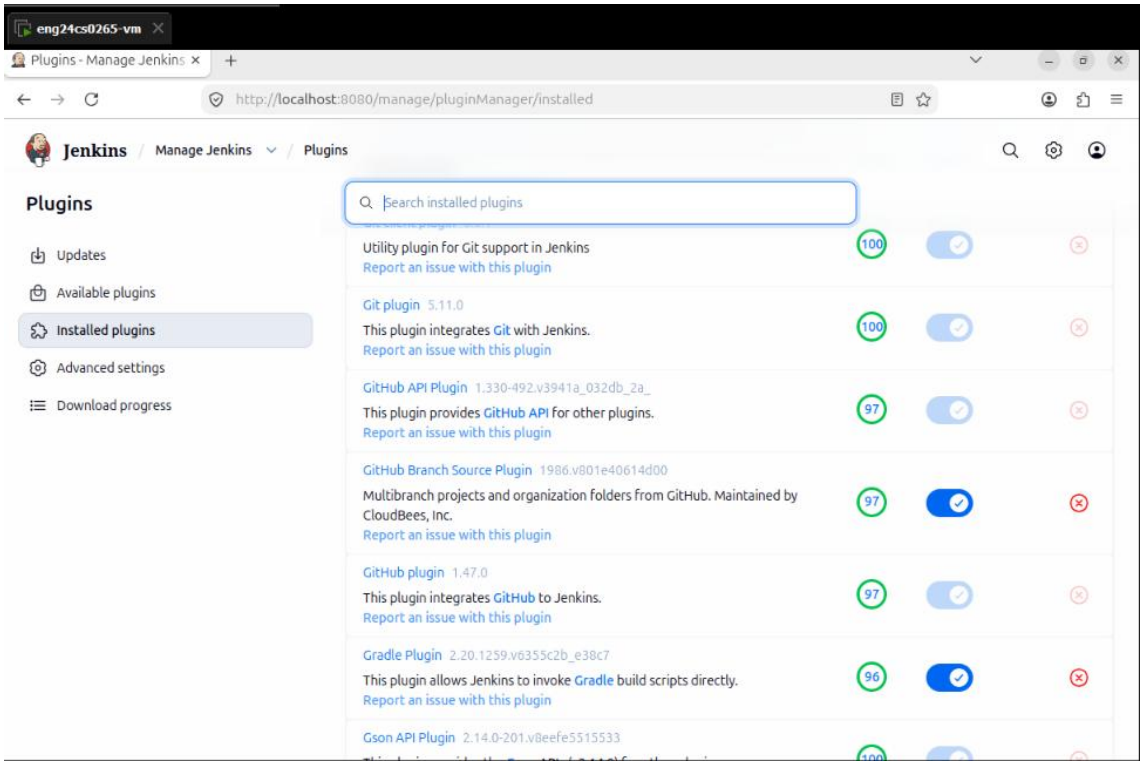
The Manage Jenkins page with System, Tools, Plugins, Nodes, Appearance, Security, Credentials and Users.

SS J3.2



Manage Jenkins > Nodes: only the Built-In Node (Linux amd64) exists, with its disk, swap and temp-space monitors; the dashboard shows it has 2 executors.

SS J3.3



Manage Jenkins > Plugins > Installed plugins, including the Git plugin 5.11.0, GitHub plugin, GitHub Branch Source and Gradle plugins.

Write: In one line each, explain controller, node, executor and plugin using what you saw.

Answer: Controller: the main Jenkins server I open at localhost:8080; it shows the web interface, stores the configuration and schedules builds. Node: a machine that runs builds; I only have the Built-In Node, so the controller also does the building (SS J3.2). Executor: one build slot on a node; my node has 2, so two builds can run at the same time (0/2 busy in SS J2.3). Plugin: an add-on that gives Jenkins a feature, for example the Git plugin that lets a job check out code from GitHub (SS J3.3).

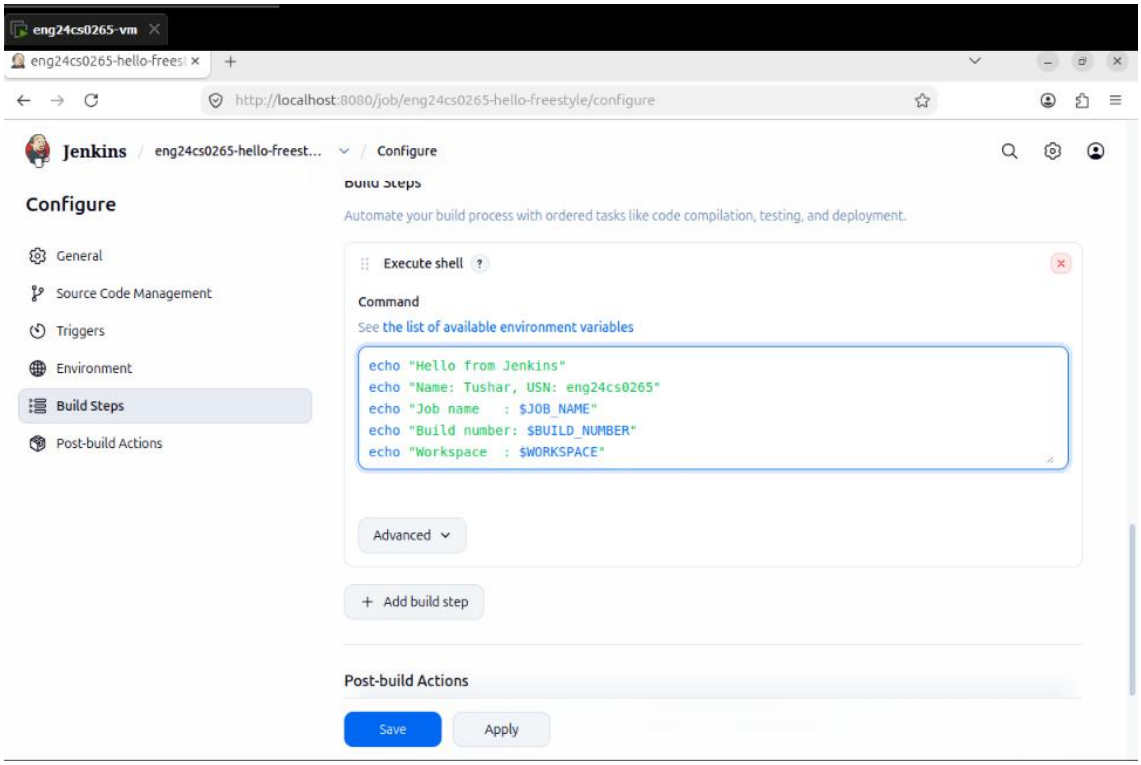
Part 2 - Freestyle jobs

Experiment J4 - Your first freestyle job

Aim: Create a job that runs shell commands and read its console output.

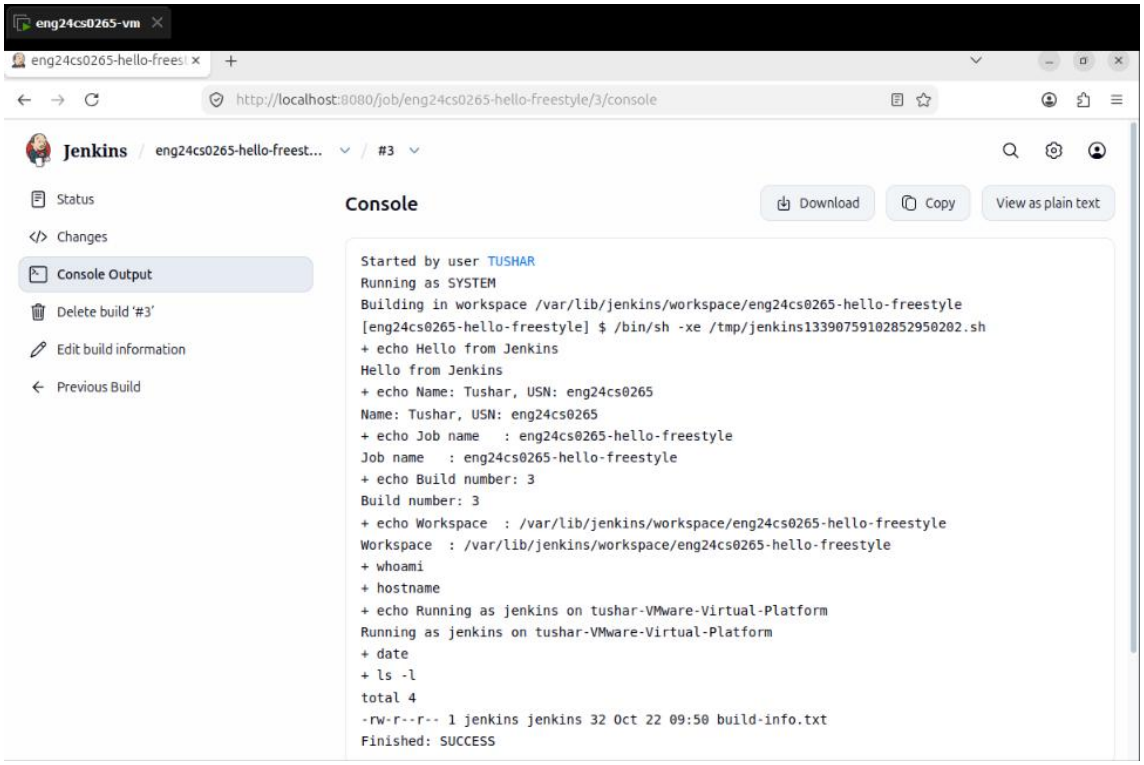
```
echo "Hello from Jenkins" ; echo "Name: Tushar, USN: eng24cs0265"
echo "Job name: $JOB_NAME" ; echo "Build number: $BUILD_NUMBER" ; echo "Workspace: $WORKSPACE"
echo "Running as : $(whoami) on $(hostname)" ; date > build-info.txt ; ls -l
```

SS J4.1



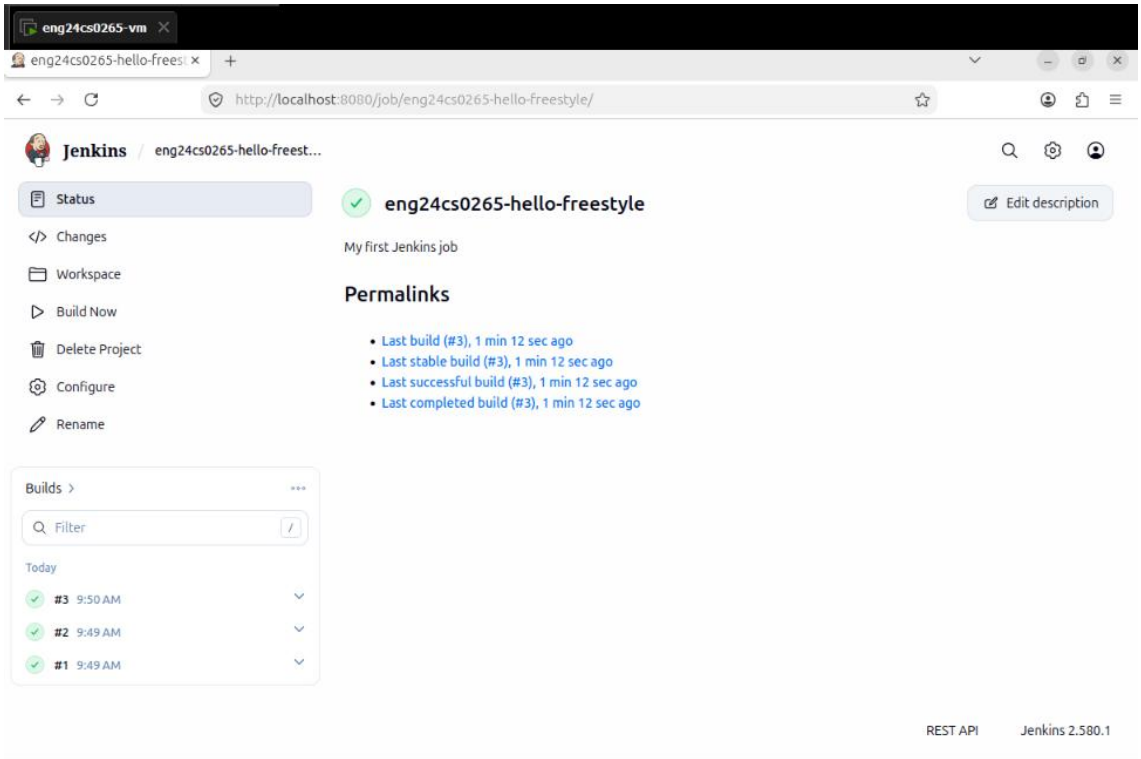
Configuration of eng24cs0265-hello-freestyle: the Execute shell build step with my echo commands that print my name, USN, job name, build number and workspace.

SS J4.2



Console Output of build #3 ending with "Finished: SUCCESS". It shows the job name, build number 3, the workspace path and "Running as jenkins on tushar-VMware-Virtual-Platform", and ls -l lists build-info.txt.

SS J4.3



The job page eng24cs0265-hello-freestyle with description "My first Jenkins job" and the Builds history list containing builds #1, #2 and #3, all green.

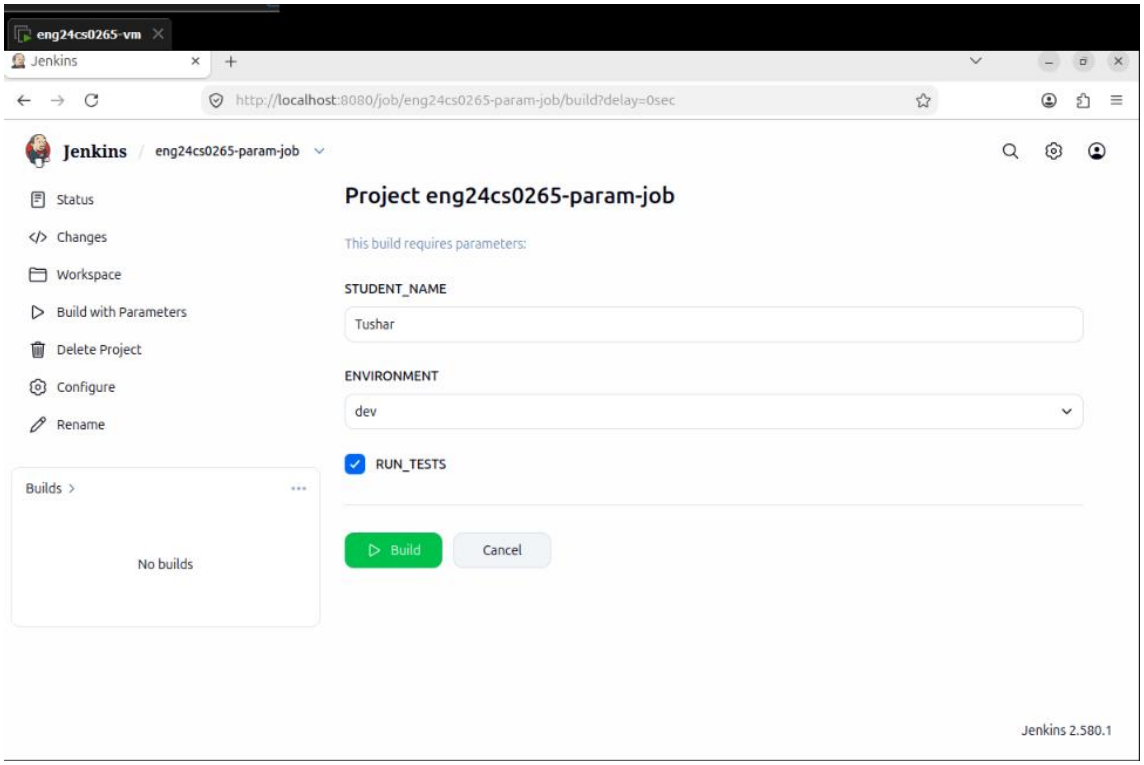
Write: Which user ran the commands? Why is build-info.txt still in the workspace after the build finished?

Answer: The commands ran as the Linux user jenkins ("Running as jenkins on ..." in SS J4.2); Jenkins itself only labels the security context as SYSTEM. build-info.txt is still in the workspace because the workspace folder /var/lib/jenkins/workspace/eng24cs0265-hello-freestyle is kept between builds; Jenkins does not clean it after a build unless a clean-up is configured.

Experiment J5 - Parameterized job

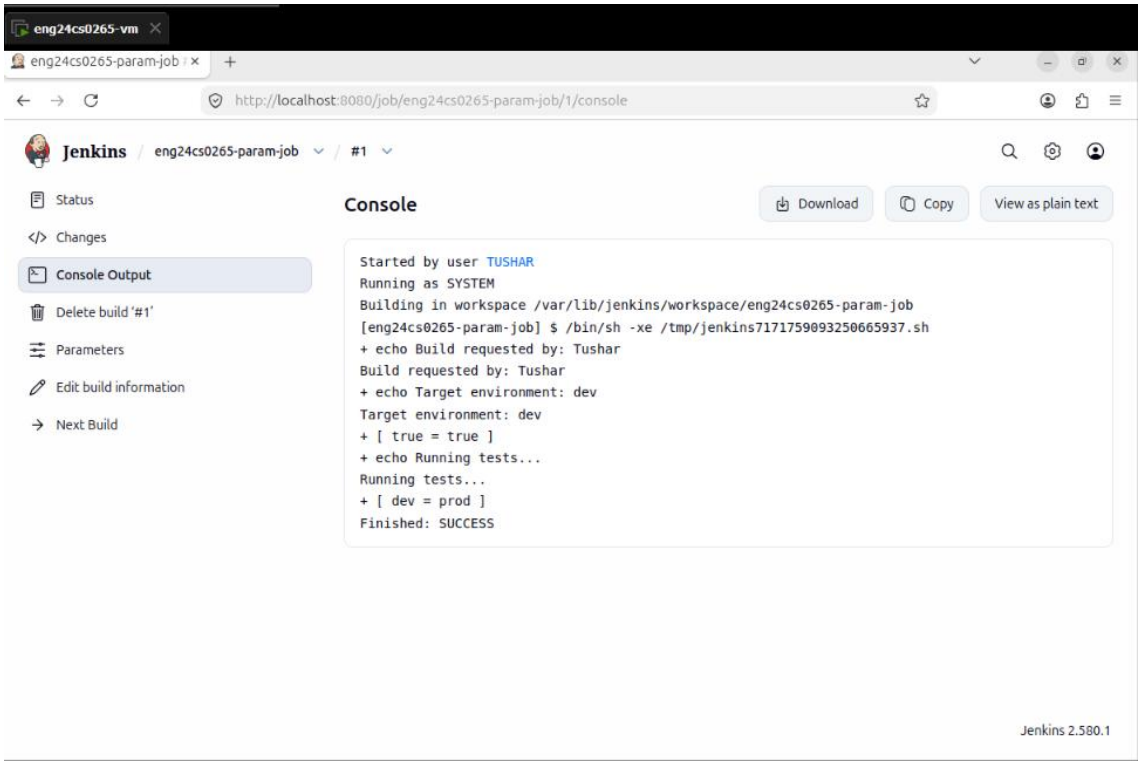
Aim: Let the person starting the build give input values.

SS J5.1



Build with Parameters page of eng24cs0265-param-job: STUDENT_NAME (Tushar), ENVIRONMENT drop-down (dev) and the RUN_TESTS tick box (ticked).

SS J5.2



Console Output of build #1 (dev, tests ticked): "Build requested by: Tushar", "Target environment: dev", "Running tests..." and Finished: SUCCESS. The second build (prod, tests unticked) was run as build #2.

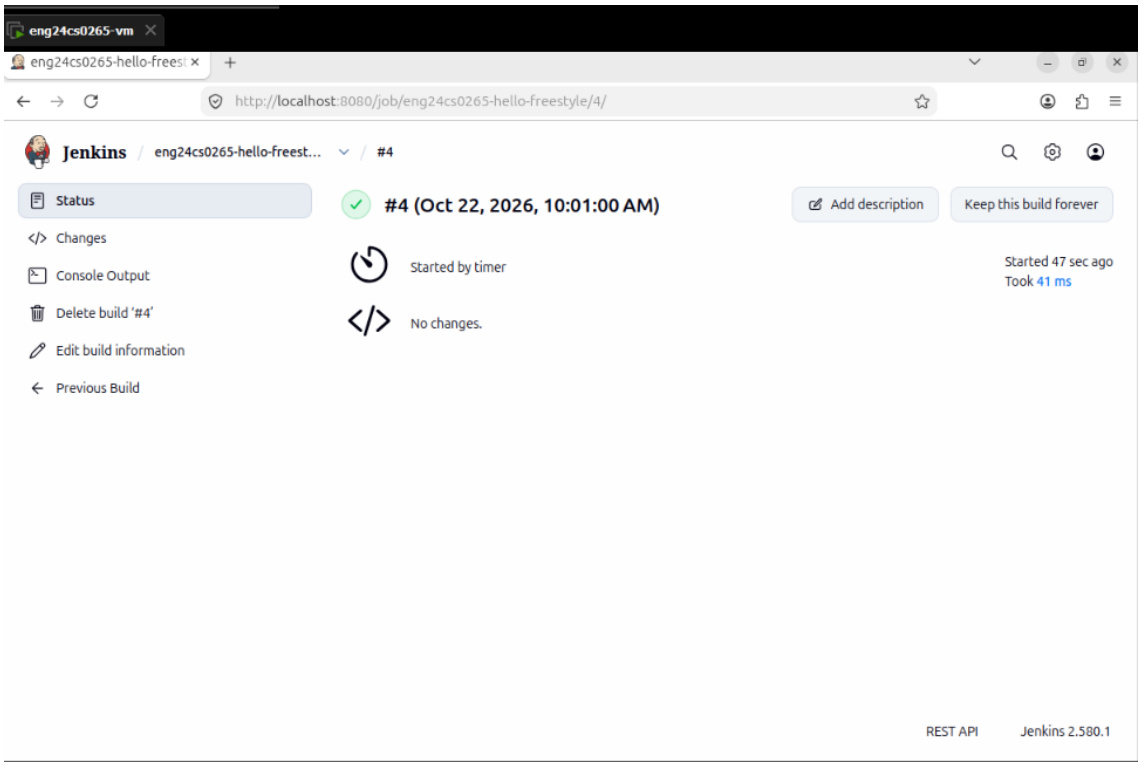
Write: Give one real situation where a parameterized build is useful.

Answer: A parameterized build is useful when the same job is used for different targets, for example a deployment job where the tester picks the environment (dev, test or prod) or the version to deploy each time, instead of keeping three copies of the job.

Experiment J6 - Build triggers: timer and job chaining

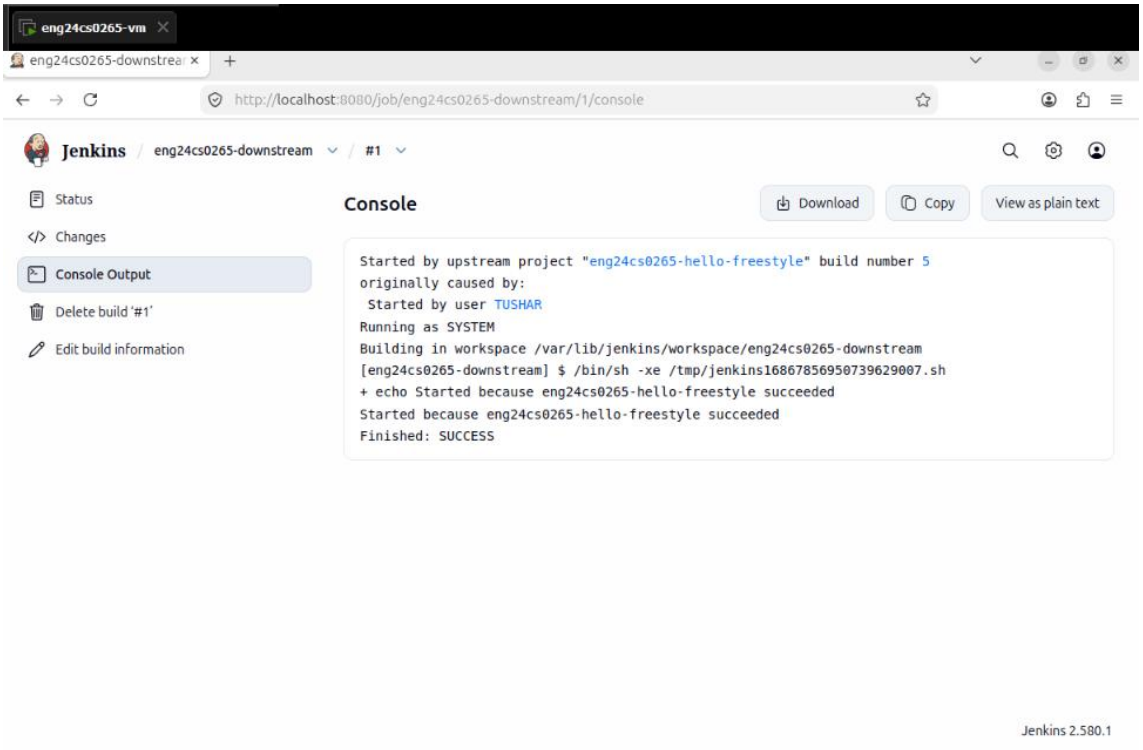
Aim: Start builds automatically instead of clicking Build Now.

SS J6.1



*Build #4 of eng24cs0265-hello-freestyle at 10:01:00 AM with the cause "Started by timer", created by the Build periodically trigger H/5 * * * *.*

SS J6.2



Console Output of eng24cs0265-downstream #1: "Started by upstream project eng24cs0265-hello-freestyle build number 5" and my echo message, Finished: SUCCESS.

Write: Write the cron expression for “every day at 11 p.m.” and for “every Monday at 9 a.m.”. Explain what upstream and downstream mean

Answer: Every day at 11 p.m.: 0 23 * * * (or H 23 * * * so Jenkins picks the minute). Every Monday at 9 a.m.: 0 9 * * 1 (or H 9 * * 1). An upstream job is the one that runs first (eng24cs0265-hello-freestyle); a downstream job is started automatically when the upstream job finishes successfully (eng24cs0265-downstream).

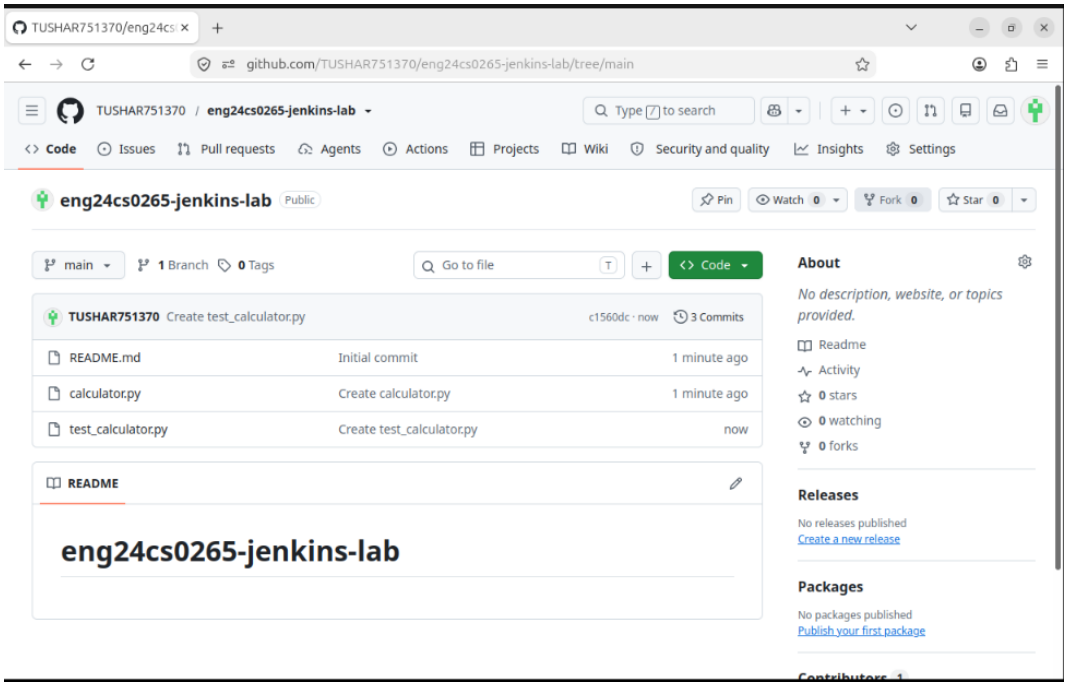
Part 3 - Jenkins with GitHub and pipelines

Experiment J7 - Create a GitHub repository and build from it

Aim: Make Jenkins fetch code from GitHub and test it on every change.

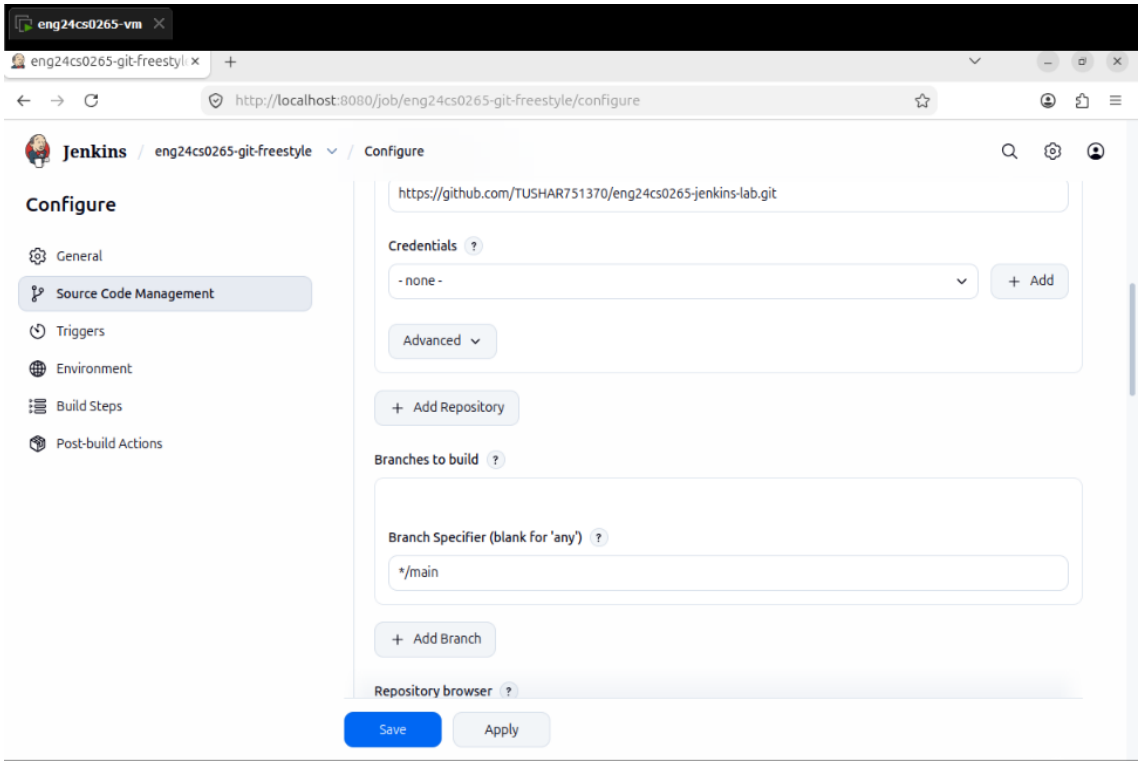
```
ls -l
python3 -m pytest -v
```

SS J7.1



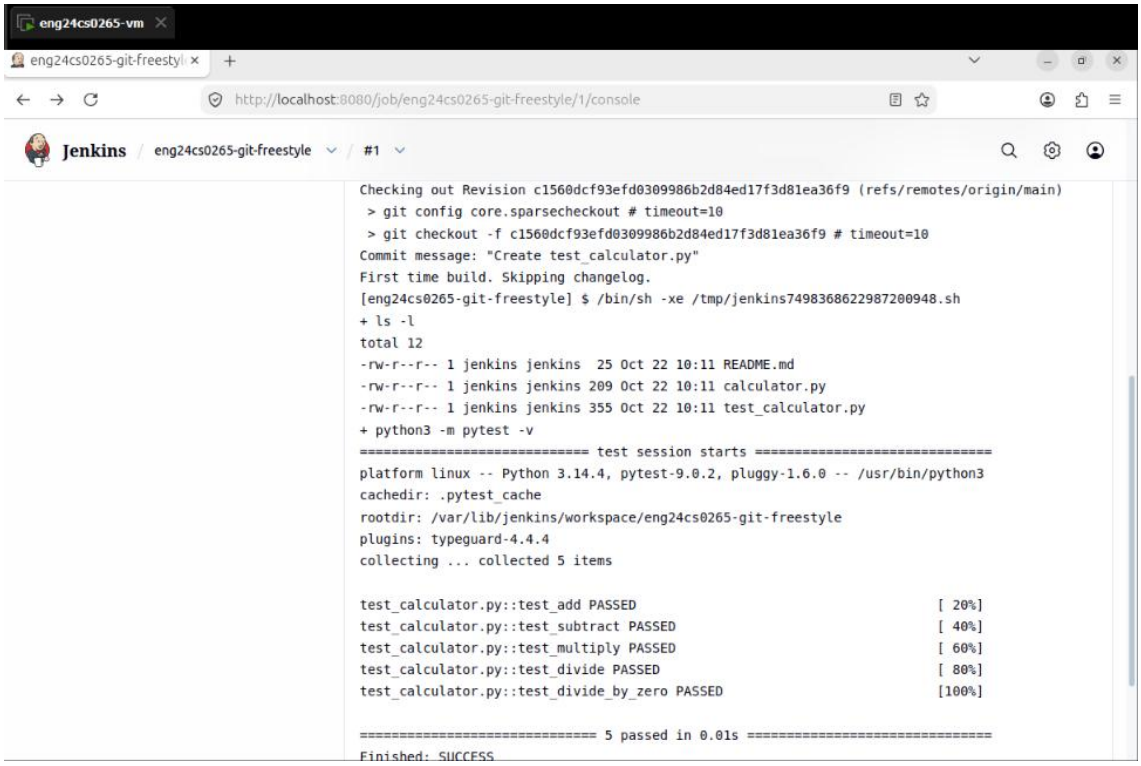
GitHub repository eng24cs0265-jenkins-lab (public, user TUSHAR751370) with the three files README.md, calculator.py and test_calculator.py on branch main.

SS J7.2



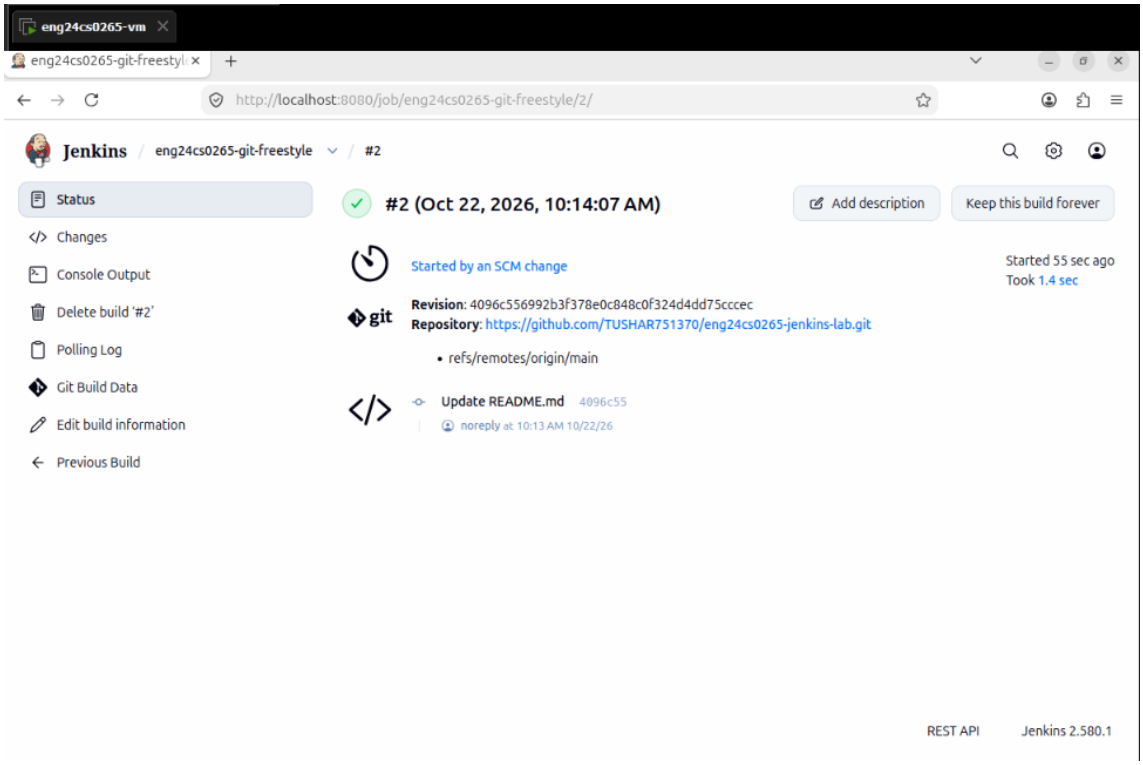
*Source Code Management of eng24cs0265-git-freestyle: Git repository URL https://github.com/TUSHAR751370/eng24cs0265-jenkins-lab.git, credentials none, Branch Specifier */main.*

SS J7.3



Console Output of build #1: Git checks out the latest commit "Create test_calculator.py", ls -l shows the three files, and pytest reports 5 passed in 0.01s, Finished: SUCCESS.

SS J7.4



Build #2 of the Git job with the cause "Started by an SCM change": the automatic build for the commit "Update README.md", with the repository and revision shown.

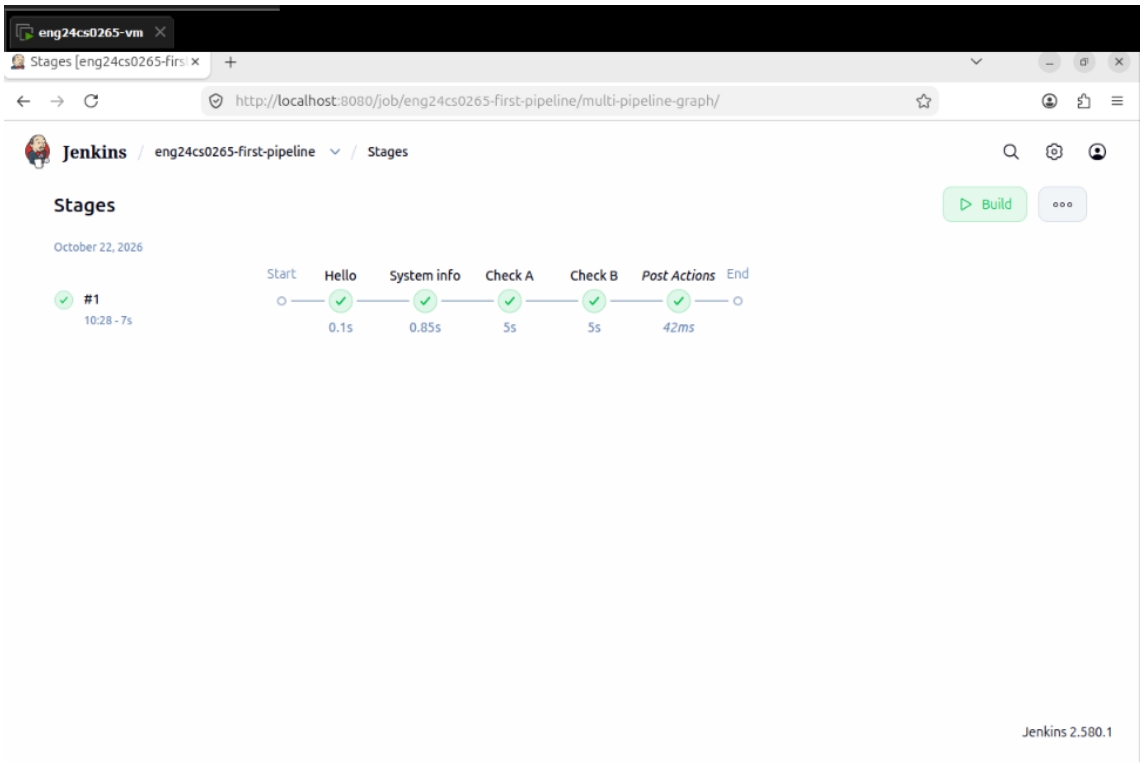
Write: Your VM is behind NAT, so we used Poll SCM. In companies, GitHub usually pushes a notification to Jenkins using a webhook. Explain the difference between polling and a webhook.

Answer: With polling (Poll SCM) Jenkins asks GitHub every few minutes whether anything changed, so a build starts with a delay and many checks find nothing; it works behind NAT because Jenkins only makes outgoing requests. With a webhook GitHub sends a notification to Jenkins immediately after each push, so the build starts at once without wasted checks, but GitHub must be able to reach the Jenkins server over the internet.

Experiment J8 - Your first pipeline (script written in Jenkins)

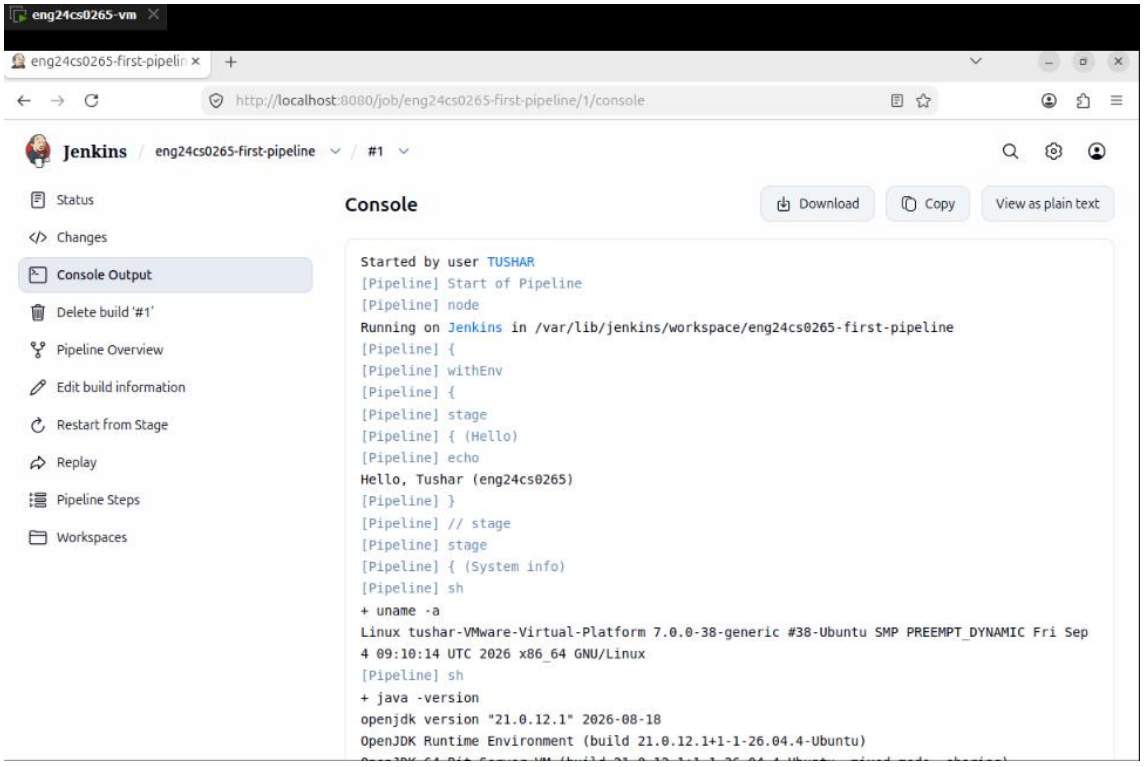
Aim: Learn declarative pipeline syntax: pipeline, agent, stages, stage, steps, post.

SS J8.1



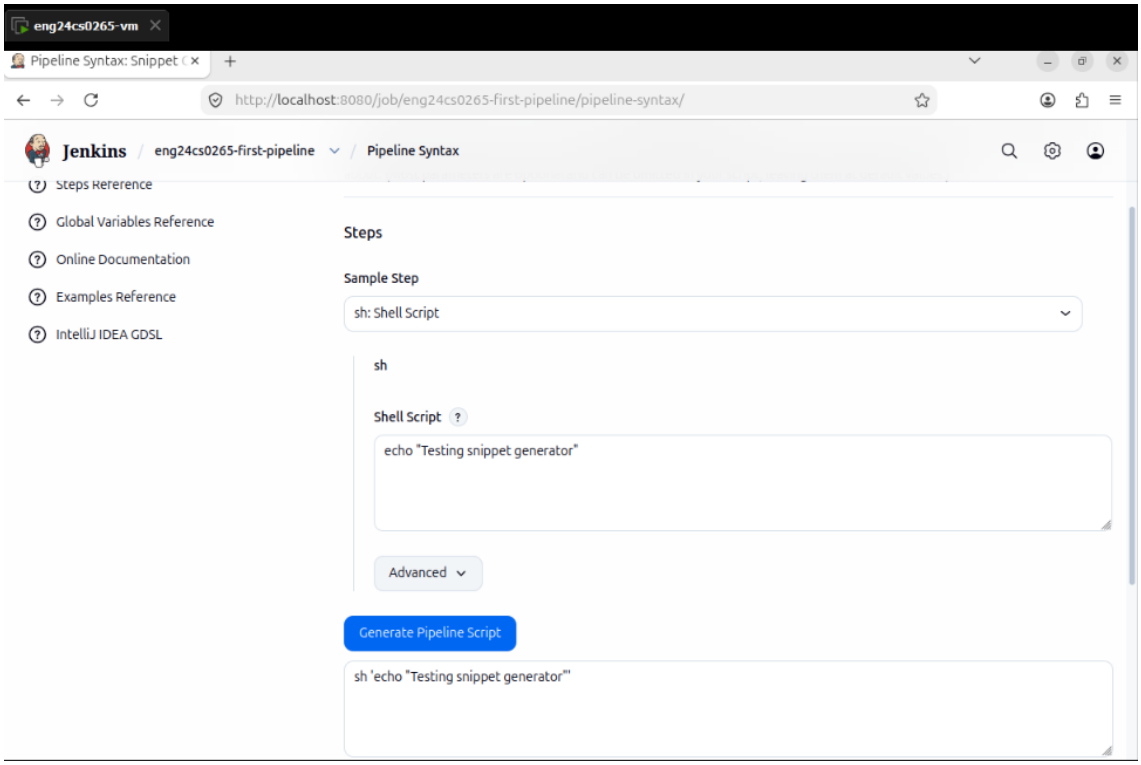
Stage view of eng24cs0265-first-pipeline build #1 (7 s): Hello, System info, Check A, Check B and Post Actions, all green. Check A and Check B took 5 s each.

SS J8.2



Console Output of the pipeline: "Hello, Tushar (eng24cs0265)" from the Hello stage, then uname -a and java -version from the System info stage.

SS J8.3



Pipeline Syntax / Snippet Generator: sample step "sh: Shell Script" with my text, and the generated code sh 'echo "Testing snippet generator"'.

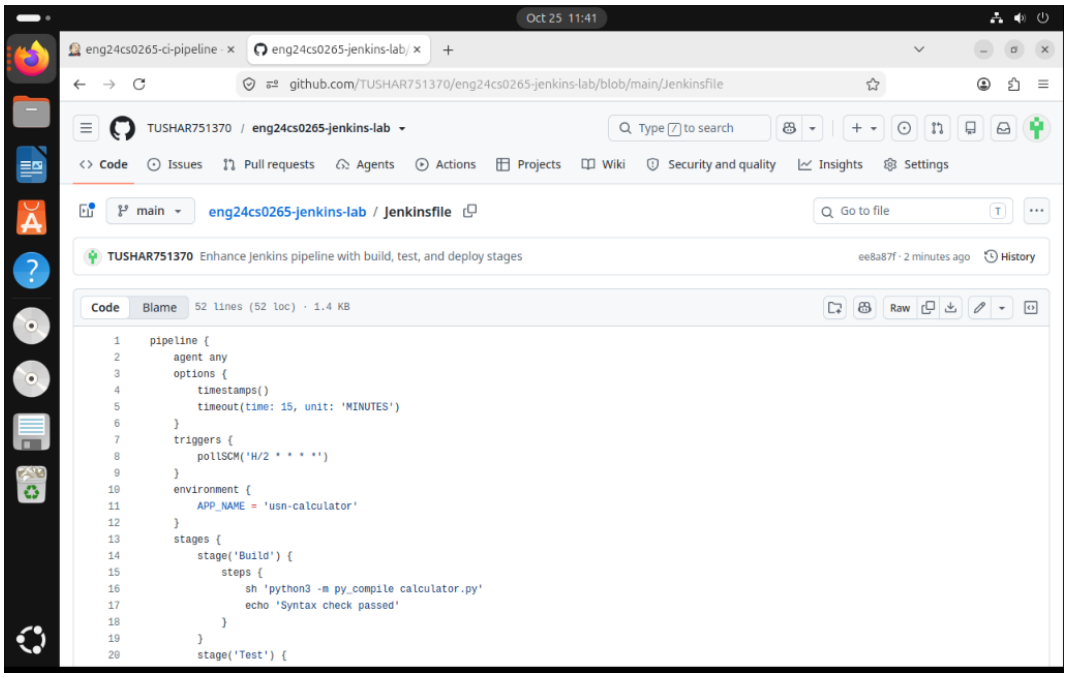
Write: Explain each keyword: pipeline , agent , environment , stages , stage , steps , parallel , post . How long did Parallel checks take, and why not 10 seconds?

Answer: pipeline is the top-level block of a declarative pipeline. agent says where it runs (any available node). environment defines variables such as STUDENT for all stages. stages holds the list of stages; a stage is one named phase (Hello, System info); steps are the actual commands inside a stage (echo, sh). parallel runs several stages at the same time (Check A and Check B). post defines actions that run after the stages, such as always, success or failure messages. The two parallel checks took about 5 seconds each in the stage view, not 10, because both ran at the same time (each sleeps 5 s), so the total is about the longest one, not their sum; the whole build took 7 s including the other short stages.

Experiment J9 - Full CI/CD pipeline from a Jenkinsfile

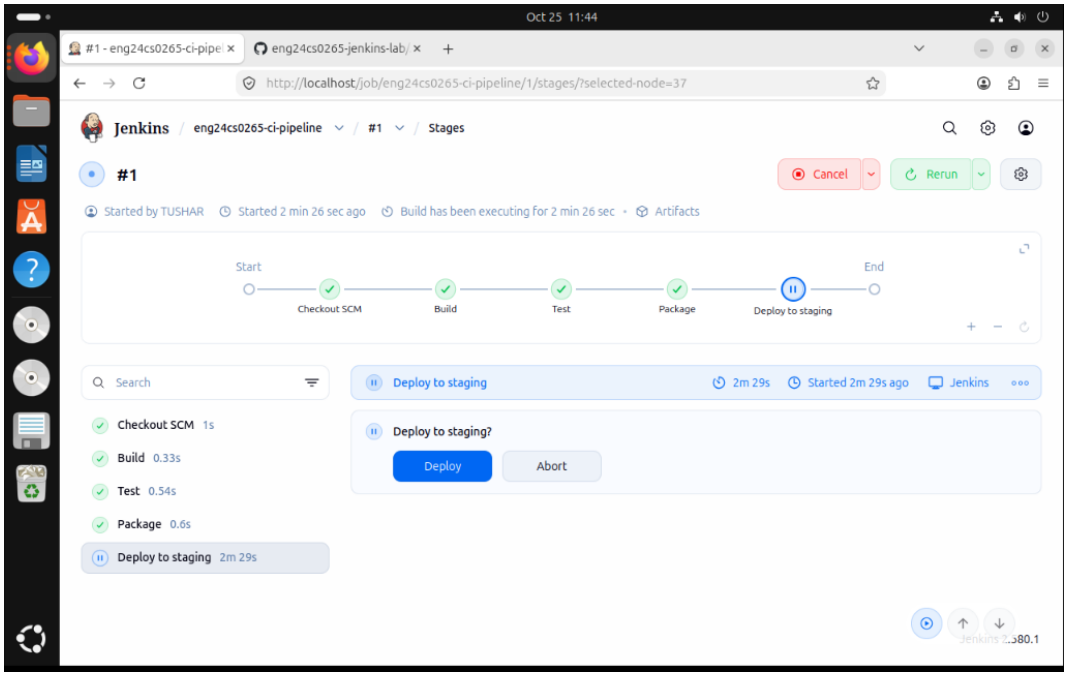
Aim: Keep the pipeline in GitHub ("pipeline as code") and run build, test and package.

SS J9.1



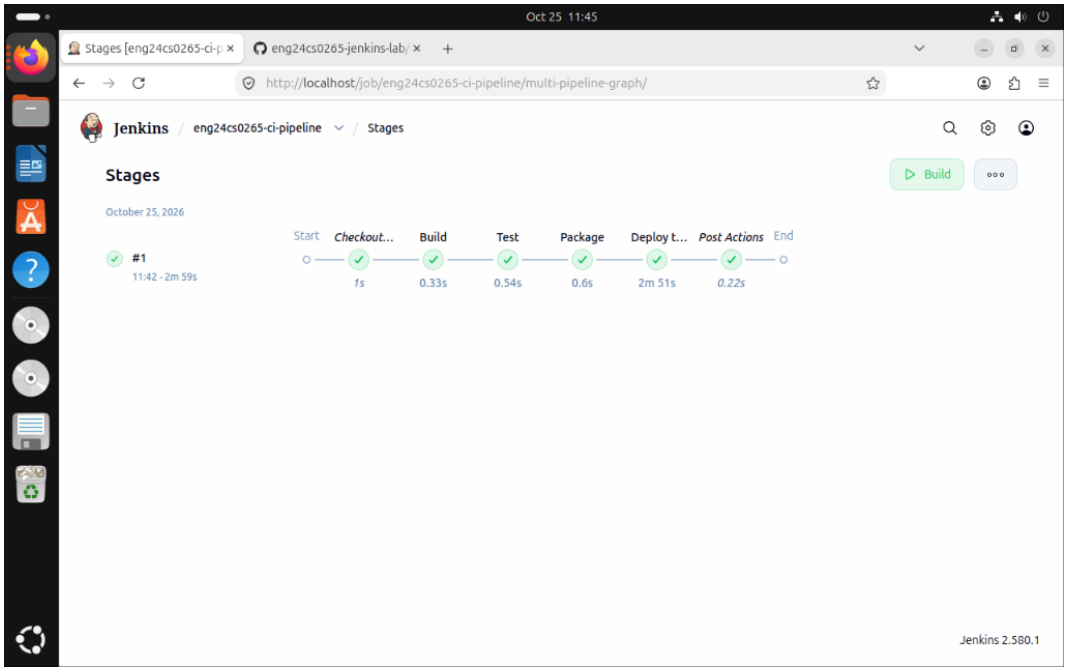
*The Jenkinsfile in my GitHub repository eng24cs0265-jenkins-lab (commit "Enhance Jenkins pipeline with build, test, and deploy stages", 52 lines): declarative pipeline with agent any, options (timestamps, 15-minute timeout), pollSCM H/2 * * * *, environment APP_NAME and the stages Build (py_compile), Test (pytest with JUnit XML), Package and Deploy to staging.*

SS J9.2



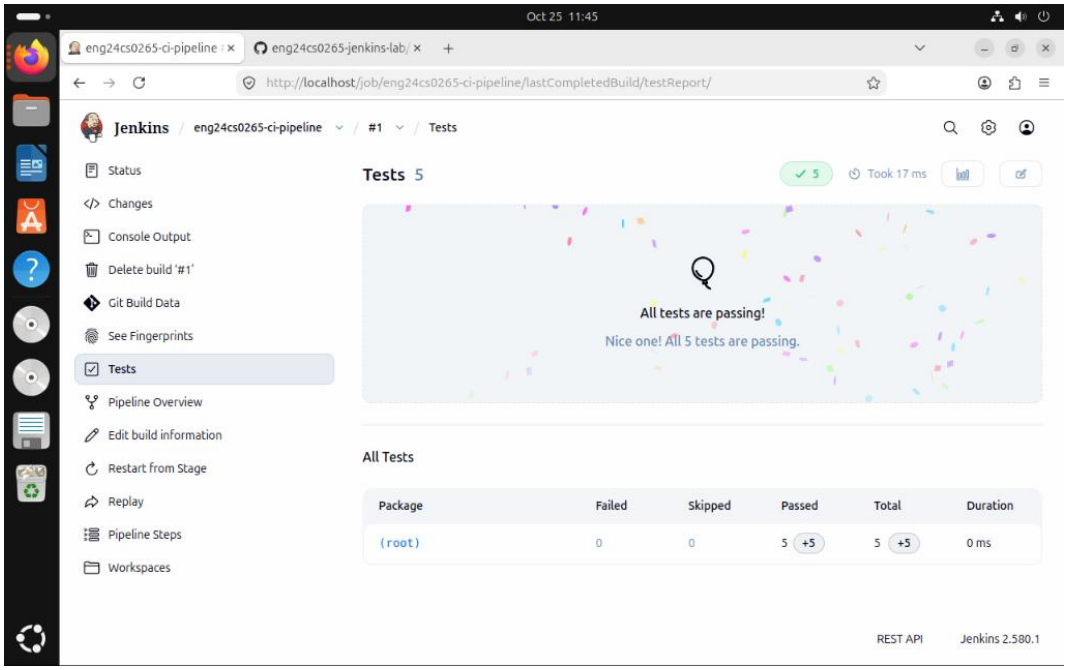
Build #1 of eng24cs0265-ci-pipeline (started by TUSHAR): Checkout SCM, Build, Test and Package are green and the pipeline is paused at Deploy to staging with the question "Deploy to staging?" and the Deploy / Abort buttons, waiting for manual approval.

SS J9.3



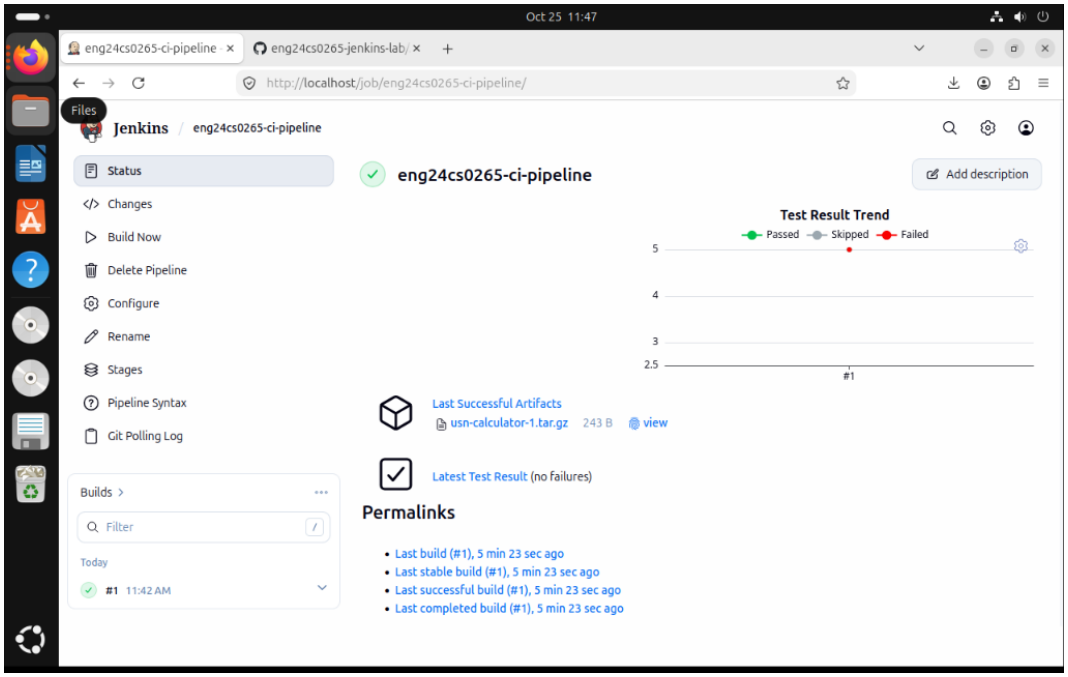
Stage view of build #1 (2 min 59 s) after the approval was given: Checkout SCM, Build, Test, Package, Deploy to staging and Post Actions are all green. Deploy to staging took 2 min 51 s because it waited for my click.

SS J9.4



Test Result page of build #1: "All tests are passing!" with 5 tests, 0 failed and 0 skipped, read by the junit step from test-results.xml (pytest --junitxml).

SS J9.5



Job page of eng24cs0265-ci-pipeline: the archived artifact usn-calculator-1.tar.gz (243 B) under Last Successful Artifacts, the Test Result Trend graph and Latest Test Result (no failures).

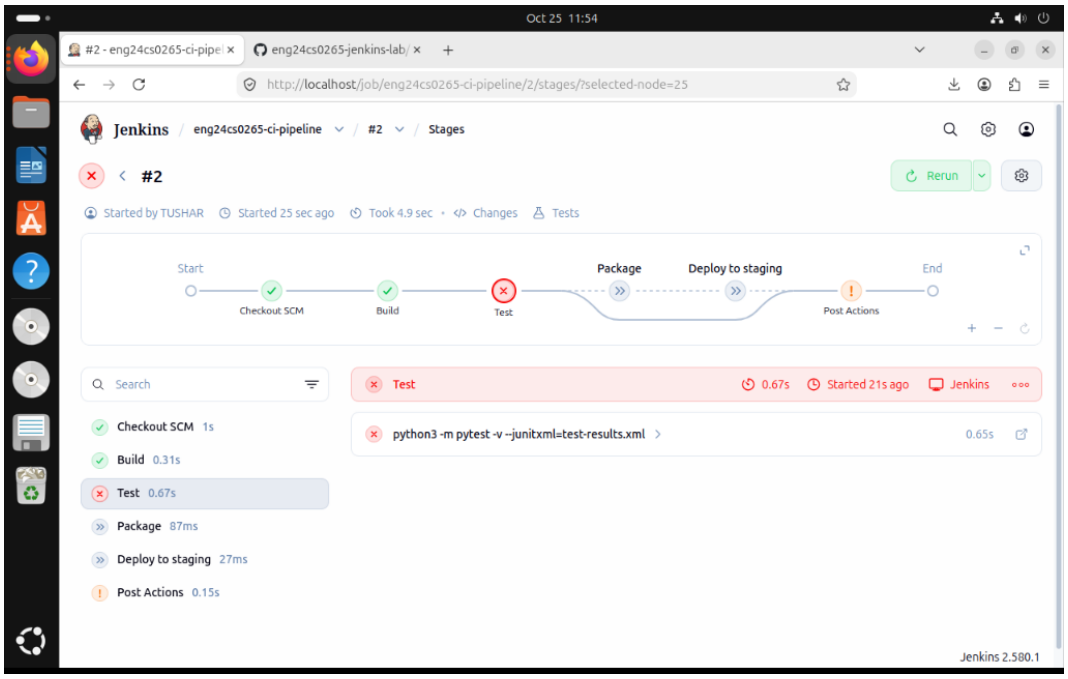
Write: Why is it better to keep the pipeline in a Jenkinsfile in Git than to type it into Jenkins (J8)? What is the purpose of the input step, and how does it relate to Continuous Delivery?

Answer: A Jenkinsfile stored in Git is versioned with the application: every change to the pipeline is in the history, can be reviewed and rolled back, and the pipeline is rebuilt together with the code on any Jenkins server. A script typed into Jenkins (J8) lives only inside Jenkins, has no history and is lost if that Jenkins is lost. The input step pauses the pipeline until a person clicks Deploy. This is the manual approval gate of Continuous Delivery: the build is always tested, packaged and ready, but a human decides when it goes to staging or production.

Experiment J10 - Break the build, then fix it

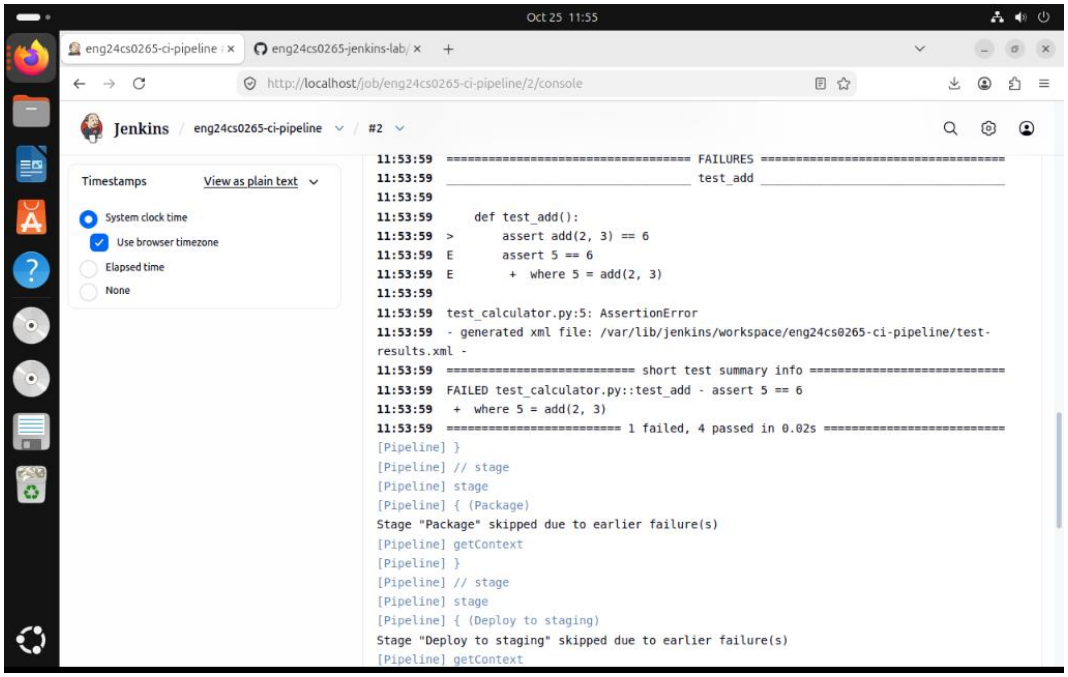
Aim: See how CI catches a bad change and how to find the cause.

SS J10.1



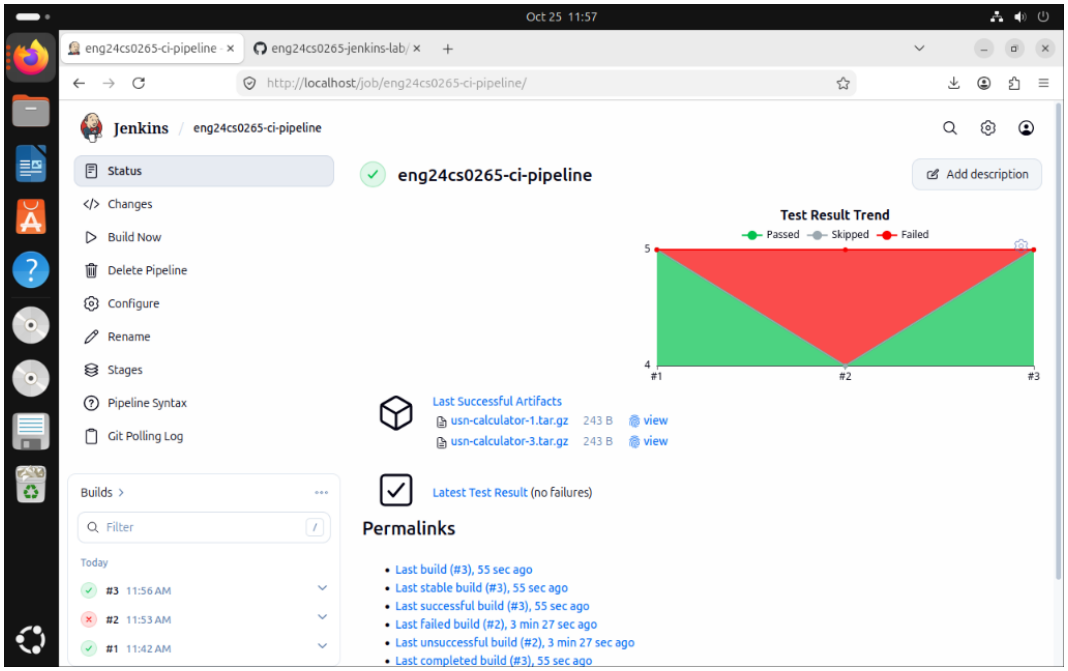
Stage view of failed build #2 (4.9 s): Checkout SCM and Build are green, Test is red (python3 -m pytest -v --junitxml=test-results.xml failed), Package and Deploy to staging are skipped (arrows) and Post Actions still ran.

SS J10.2



Console Output of failed build #2: test_add FAILED, "assert 5 == 6" with "where 5 = add(2, 3)", and the summary "1 failed, 4 passed in 0.02s"; the stages Package and Deploy to staging are reported as skipped due to earlier failure(s). I changed the expected value in test_calculator.py to 6 on purpose.

SS J10.3



Job page after the fix: the Builds list shows #3 (green), #2 (red) and #1 (green). The Test Result Trend graph drops to 4 passed at #2 and recovers at #3, and usn-calculator-1.tar.gz and usn-calculator-3.tar.gz are archived (no artifact for the failed build #2).

Write: Which stages ran in the failed build, and why did the pipeline stop where it did? How does Jenkins show whose commit broke the build, and why is finding this out within 2 minutes valuable?

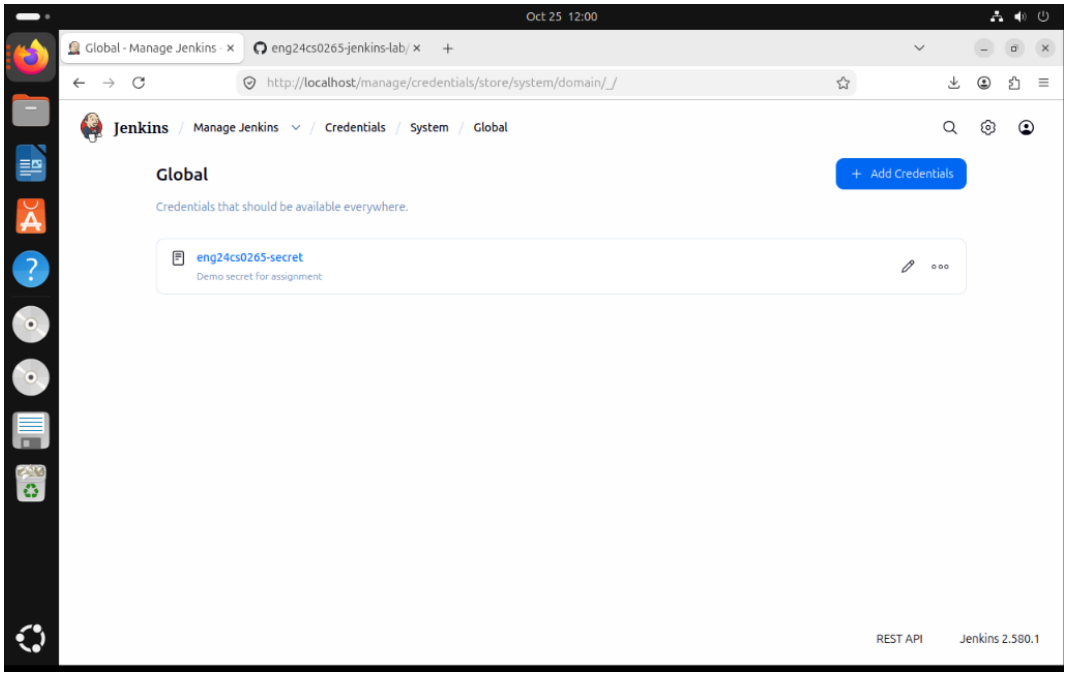
Answer: In the failed build the Checkout SCM and Build stages passed, then Test failed, and Package and Deploy to staging were skipped. A pipeline stops at the first failing stage so that code with a failing test is never packaged or deployed. Jenkins records the commit that started each build (the Changes link of build #2 shows the commit message and author), so I can see which push broke build #2. Finding this within about 2 minutes of the push (Jenkins polls every 2 minutes and the failed build itself took only 4.9 s) is valuable because only one small change is to blame, the author still remembers it, and the problem is fixed before others build on top of it.

Part 4 - Credentials, Docker and administration

Experiment J11 - Store and use a secret with Credentials

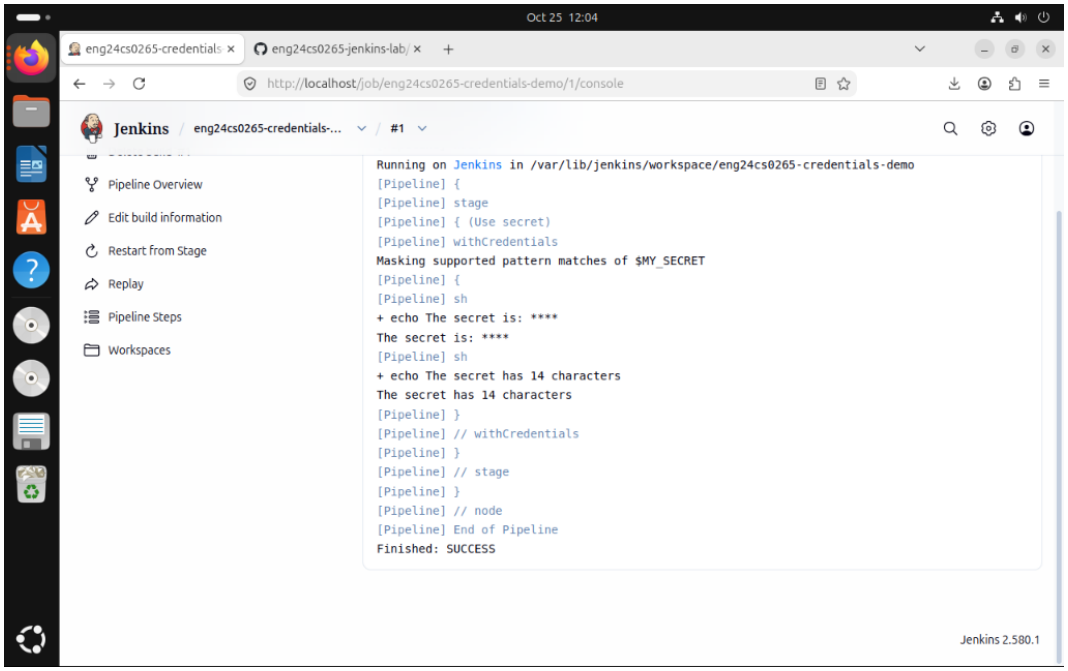
Aim: Use a password or token in a build without exposing it.

SS J11.1



Manage Jenkins > Credentials > System > Global: the secret eng24cs0265-secret ("Demo secret for assignment"); its value is not shown.

SS J11.2



*Console Output of build #1 of eng24cs0265-credentials-demo: withCredentials masks $MY_SECRET, so the log prints "The secret is: ****" and "The secret has 14 characters", then Finished: SUCCESS.*

Write: Why should passwords and API tokens never be written directly in a Jenkinsfile in GitHub?

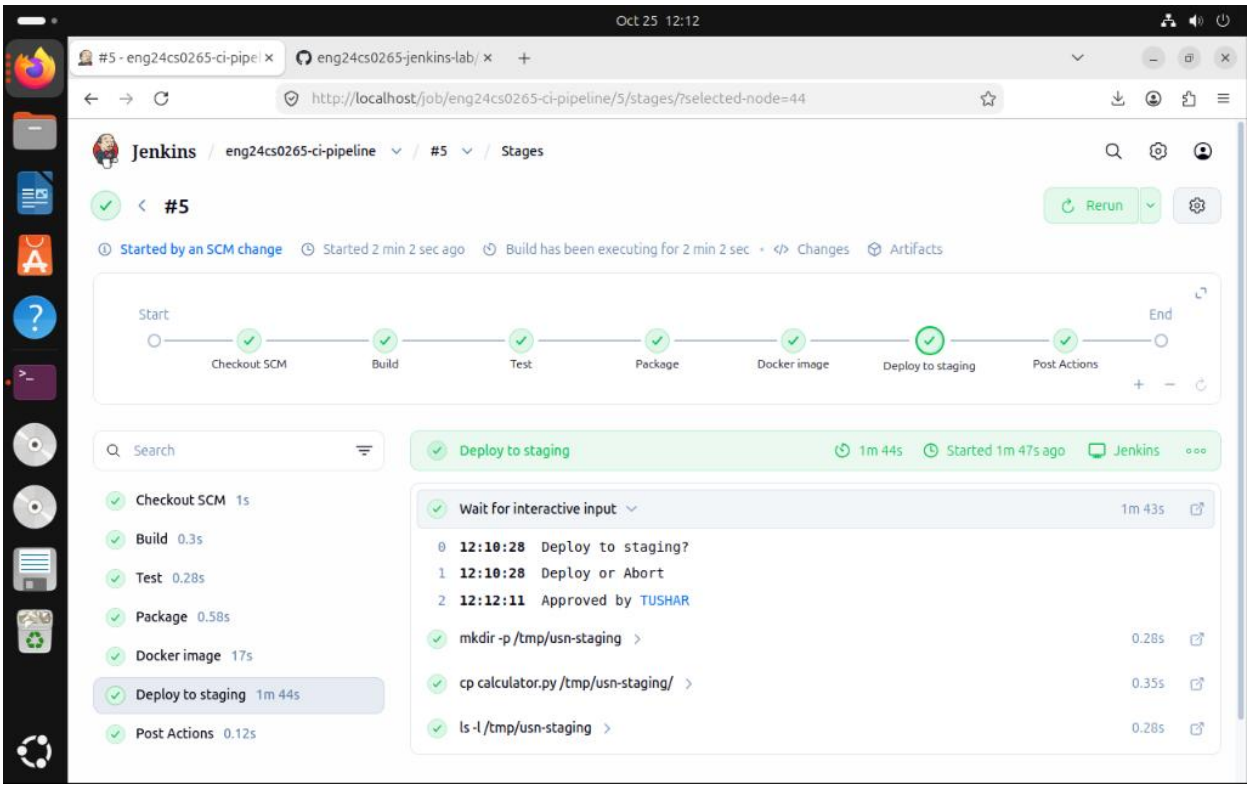
Answer: Passwords and API tokens must not be written in a Jenkinsfile in GitHub because anyone who can read the repository can read them, and they stay in the Git history forever even if the line is deleted later. On a public repository they would be visible to everyone. Jenkins Credentials stores the secret encrypted, the pipeline only refers to its ID, and Jenkins masks the value as **** in the logs.

Experiment J12 - Build and run a Docker image from the pipeline

Aim: Combine Jenkins with Docker (from the Docker assignment).

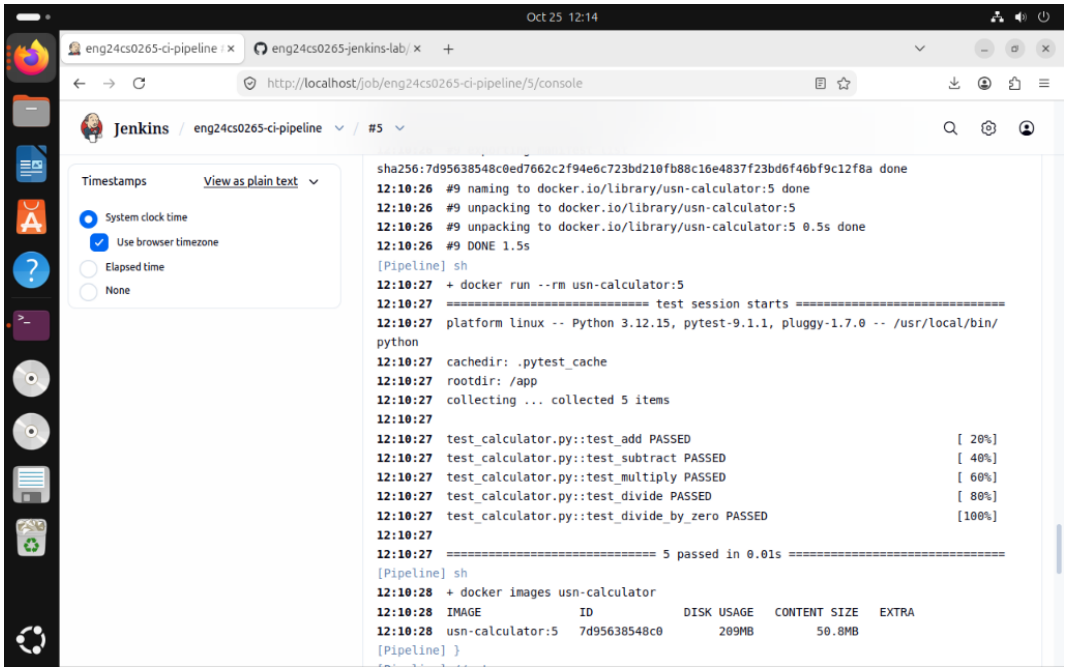
In my pipeline the Docker integration is a separate stage, Docker image, in the Jenkinsfile: it builds an image from the Dockerfile in the repository (docker build -t ${APP_NAME}:${BUILD_NUMBER} .), runs it (docker run --rm, which runs the tests inside the container) and lists it with docker images. The image is tagged with the Jenkins build number.

SS J12.1



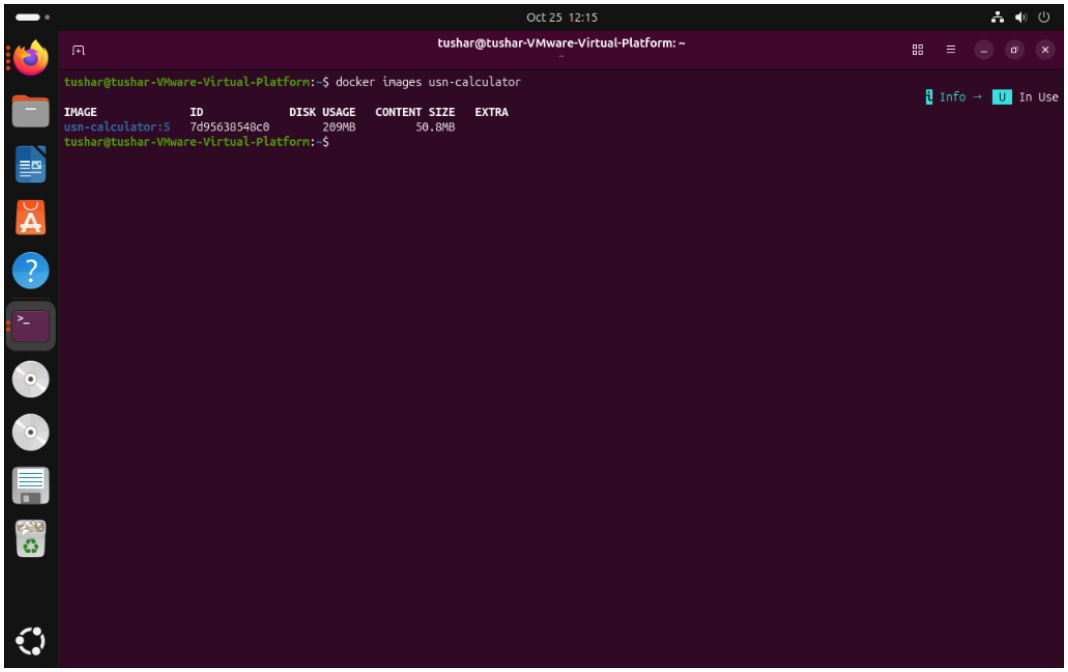
Stage view of build #5 (started by an SCM change): Checkout SCM, Build, Test, Package, Docker image (17 s), Deploy to staging and Post Actions are all green. Deploy to staging shows "Approved by TUSHAR" and then mkdir, cp and ls -l of /tmp/usn-staging.

SS J12.2



Console Output of build #5, Docker image stage: docker run --rm usn-calculator:5 runs pytest inside the container (Python 3.12.15, pytest 9.1.1) and all five tests PASSED (5 passed in 0.01s); docker images usn-calculator then lists the image usn-calculator:5 (209MB).

SS J12.3



VM terminal: docker images usn-calculator shows the image usn-calculator with tag 5, the build number of build #5 (image ID 7d95638548c0).

Write: Why do teams build a Docker image in their CI pipeline instead of copying source files to the server?

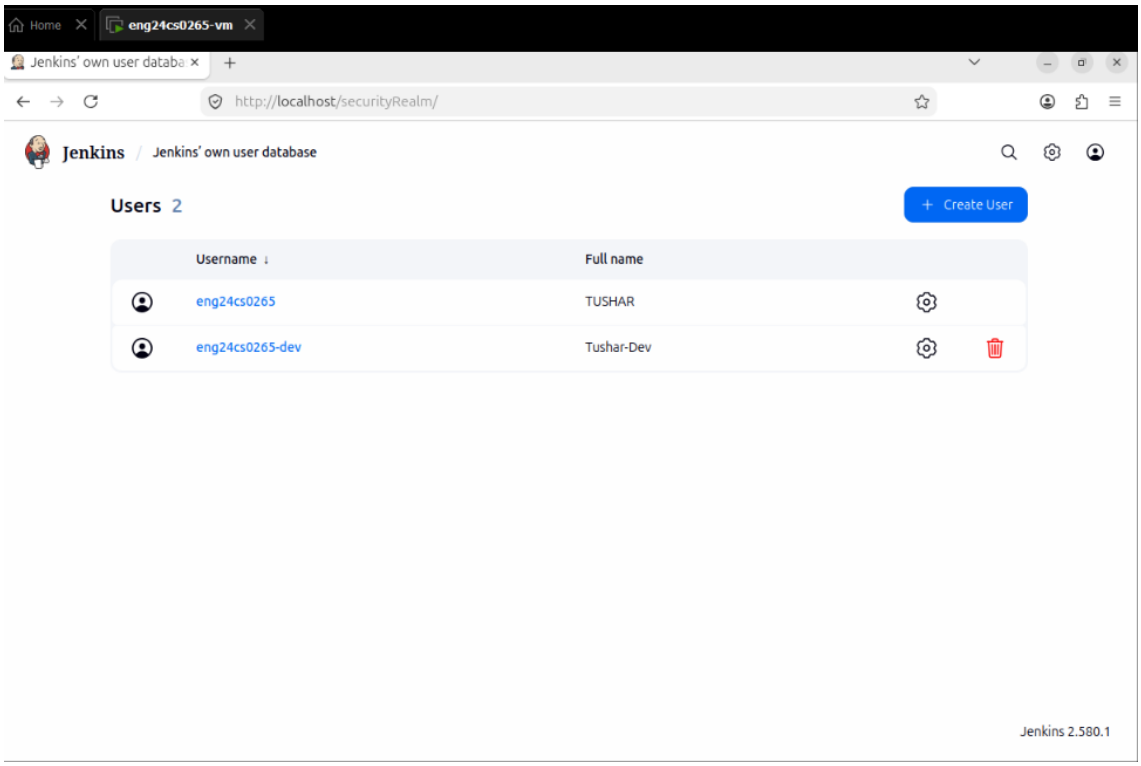
Answer: Teams build a Docker image in CI because the image contains the application together with the exact runtime and libraries, so it runs the same way on the tester's laptop, on staging and on the server. Copying only source files would depend on the server having the right Python version and packages. The image is also built once, tagged with the build number and can be rolled back to an older tag.

Experiment J13 - Users, plugins and maintenance

Aim: Do basic Jenkins administration.

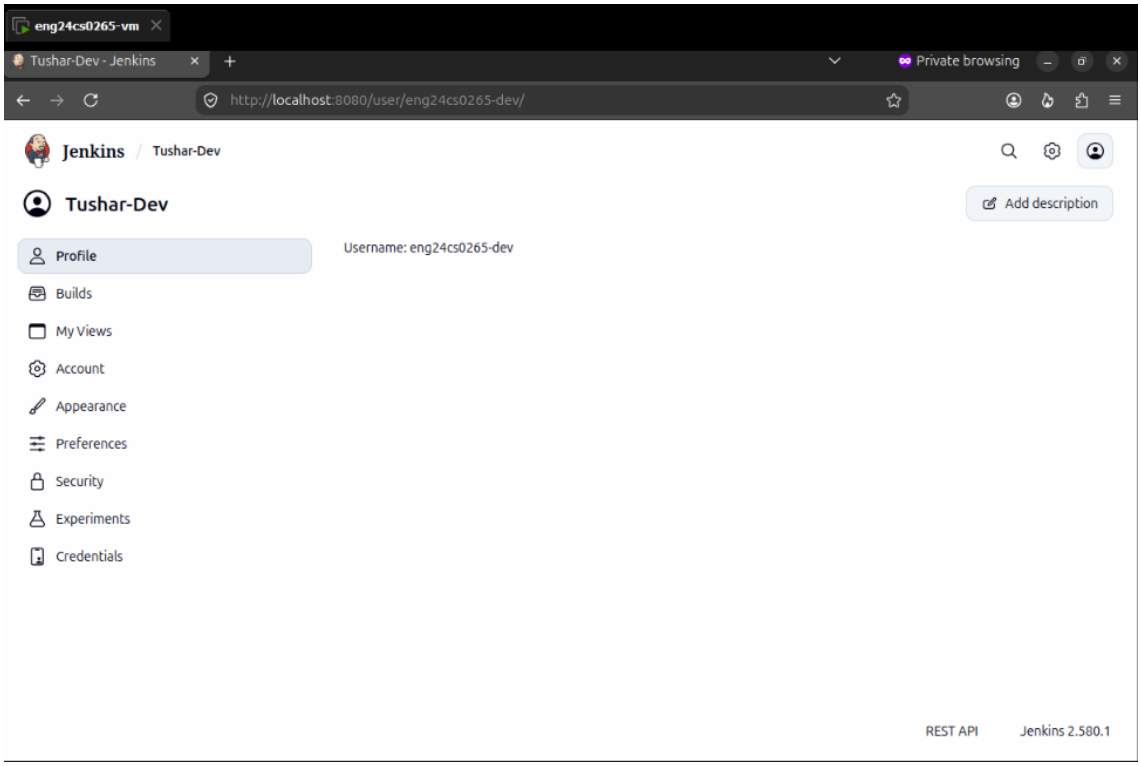
```
sudo du -sh /var/lib/jenkins
sudo ls /var/lib/jenkins/jobs
```

SS J13.1



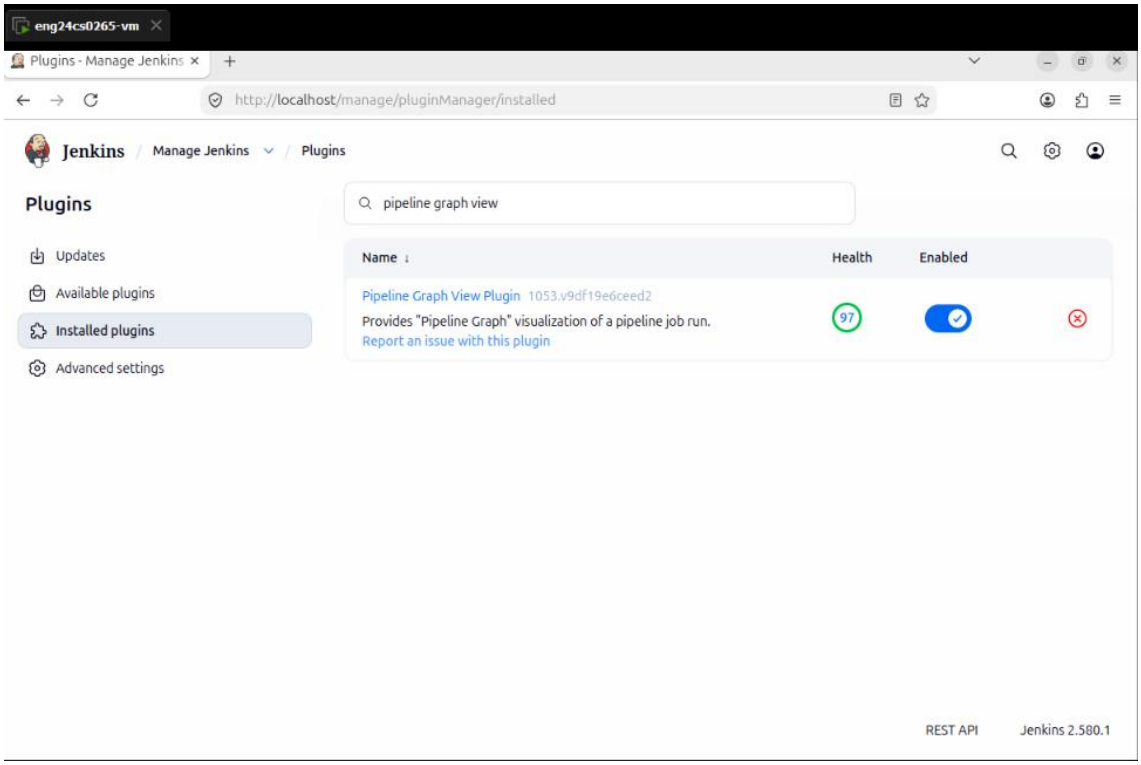
Users page ("Jenkins' own user database") listing two users: my account eng24cs0265 (TUSHAR) and the second user eng24cs0265-dev (Tushar-Dev).

SS J13.2



Firefox private window (Private browsing) logged in as the second user: profile page "Tushar-Dev" with username eng24cs0265-dev at localhost:8080/user/eng24cs0265-dev/.

SS J13.3



Manage Jenkins > Plugins > Installed plugins searching "pipeline graph view": Pipeline Graph View Plugin 1053 is installed and enabled.

SS J13.4



Terminal: sudo du -sh /var/lib/jenkins gives 190M, and sudo ls /var/lib/jenkins/jobs lists my six jobs (downstream, first-pipeline, git-freestyle, hello-freestyle, jenkinsfile-pipeline, param-job).

Write: All jobs, builds and settings live in /var/lib/jenkins . What would you back up to restore this Jenkins on a new machine?

Answer: Everything Jenkins knows is inside /var/lib/jenkins (190 MB here). To restore Jenkins on a new machine I would back up that folder, mainly config.xml and the other .xml settings, the jobs folder with the job configurations and build history, the plugins folder (or at least the list of plugins), the users folder and the secrets folder, which is needed to decrypt stored credentials. Then I would install the same Jenkins version on the new machine, stop the service, restore the folder with the right owner (jenkins) and start Jenkins again.

Final Jenkins file

The Jenkinsfile in my repository https://github.com/TUSHAR751370/eng24cs0265-jenkins-lab (branch main). This is the complete final version (59 lines), including the Docker image stage and the manual approval before Deploy to staging. The earlier 52-line version without the Docker image stage is shown in SS J9.1:

```
pipeline {
  agent any
  options {
    timestamps()
    timeout(time: 15, unit: 'MINUTES')
  }
  triggers {
    pollSCM('H/2 * * * *')
  }
  environment {
    APP_NAME = 'usn-calculator'
  }
  stages {
    stage('Build') {
      steps {
        sh 'python3 -m py_compile calculator.py'
        echo 'Syntax check passed'
      }
    }
    stage('Test') {
      steps {
        sh 'python3 -m pytest -v --junitxml=test-results.xml'
      }
    }
    stage('Package') {
      steps {
        sh 'mkdir -p dist'
        sh 'tar -czf dist/${APP_NAME}-${BUILD_NUMBER}.tar.gz calculator.py'
        archiveArtifacts artifacts: 'dist/*.tar.gz', fingerprint: true
      }
    }
    stage('Docker image') {
      steps {
        sh 'docker build -t ${APP_NAME}:${BUILD_NUMBER} .'
        sh 'docker run --rm ${APP_NAME}:${BUILD_NUMBER}'
        sh 'docker images ${APP_NAME}'
      }
    }
    stage('Deploy to staging') {
      steps {
        input message: 'Deploy to staging?', ok: 'Deploy'
        sh 'mkdir -p /tmp/usn-staging'
        sh 'cp calculator.py /tmp/usn-staging/'
        sh 'ls -l /tmp/usn-staging'
      }
    }
  }
  post {
    always {
      junit allowEmptyResults: true, testResults: 'test-results.xml'
    }
    success {
      echo "Build ${env.BUILD_NUMBER} of ${env.APP_NAME} passed"
    }
    failure {
```

```
        echo "Build ${env.BUILD_NUMBER} failed — read the Console Output"
    }
  }
}
```

Pipeline Diagram

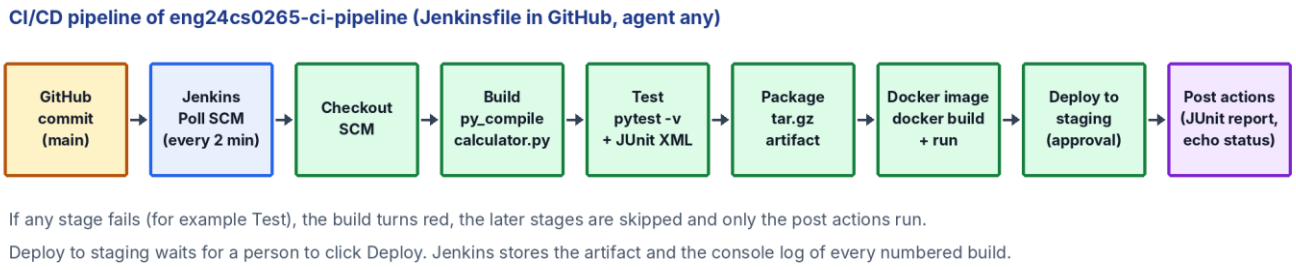


Figure: flow from a GitHub commit to the packaged result: Jenkins polls the repository, then runs Checkout SCM, Build, Test, Package, Docker image and Deploy to staging (after manual approval); the post actions run at the end.

Conclusion

In this assignment I installed Jenkins as a service in my Ubuntu VM, completed the setup wizard and explored the interface (controller, Built-In Node with 2 executors, plugins). I built freestyle jobs with shell steps, parameters, a timer trigger and an upstream/downstream chain, and then connected Jenkins to my GitHub repository so that every commit to main started a build by polling. I learned declarative pipeline syntax, kept the pipeline in a Jenkinsfile in Git with a manual approval step before Deploy to staging, and saw how CI catches a bad change: when I changed a test on purpose, the Test stage turned red, Package and Deploy to staging were skipped, and the console showed exactly which assertion failed. Finally I added a second user and a plugin, added a Docker image stage that builds and runs a tagged image of the calculator, and looked at what has to be backed up in /var/lib/jenkins.

Real Problem:- A problem I faced was that a Git job or pipeline did not find the code or did not start automatically after my commit. The cause is usually that the Branch Specifier does not match the GitHub branch name. I fixed it by checking the console output and the Git polling log and setting the branch to */main exactly as it is named on GitHub.